

Unidade III

5 MANIPULAÇÃO DE LISTAS E DICIONÁRIOS

As listas e os dicionários são duas das estruturas de dados mais importantes e versáteis em Python. Juntas, elas permitem organizar e manipular coleções de dados de forma eficiente, adaptando-se a uma ampla variedade de necessidades e contextos de programação.

Uma lista em Python é uma sequência ordenada de elementos, na qual cada item tem uma posição específica, conhecida como índice. Essa estrutura de dados é mutável, o que significa que os elementos podem ser adicionados, removidos ou modificados após a criação da lista. A flexibilidade das listas as torna adequadas para armazenar e organizar conjuntos de dados heterogêneos, nas quais os itens podem ser de tipos diferentes, como números, strings ou até mesmo outras listas. Além disso, Python fornece uma série de métodos embutidos para facilitar o trabalho com listas, como adicionar itens no final, inserir elementos em posições específicas, ordenar os valores ou mesmo reverter sua ordem. Essa riqueza de funcionalidades torna as listas uma ferramenta indispensável para manipulação de dados em Python.

Os dicionários são estruturas de dados baseadas em mapeamento, no qual os elementos são armazenados em pares de chave-valor. Cada chave é única e funciona como um identificador para acessar seu valor correspondente. Essa característica permite que os dicionários sejam ideais para representar relações entre dados, como um nome associado a um número de telefone ou palavras e suas traduções. A mutabilidade dos dicionários também os torna altamente dinâmicos, permitindo alterações nos pares de chave-valor a qualquer momento.

A principal diferença entre listas e dicionários reside na forma como os elementos são acessados. Enquanto nas listas o acesso é feito por meio de índices numéricos que indicam a posição dos elementos, nos dicionários o acesso ocorre através das chaves, que podem ser de qualquer tipo imutável, como strings, números ou tuplas. Essa diferença fundamental reflete o propósito de cada estrutura: as listas são mais adequadas para armazenar sequências de dados ordenados, enquanto os dicionários são mais apropriados para representar coleções de dados que têm associações específicas.

Ambas as estruturas são projetadas para oferecer simplicidade e eficiência, integrando-se perfeitamente com outras funcionalidades do Python. Elas permitem iteração direta, o que significa que podem ser usadas em loops para acessar e processar seus elementos de maneira fácil e intuitiva. Além disso, listas e dicionários podem ser combinados em hierarquias mais complexas, como listas de dicionários ou dicionários de listas, para atender a requisitos específicos de programação.

Em resumo, listas e dicionários são pilares da manipulação de dados em Python, cada um com características e propósitos distintos. Juntas, essas estruturas de dados fornecem ferramentas poderosas para organizar, acessar e transformar informações, adaptando-se a diferentes tipos de problemas e contextos. Dominar o uso de listas e dicionários é um passo vital para qualquer programador que deseja explorar o potencial da linguagem Python e aplicá-la em uma ampla variedade de cenários.

5.1 Listas: operações e métodos

As listas em Python permitem armazenar e manipular coleções de elementos de forma ordenada, flexível e eficiente, adaptando-se a diversas necessidades de programação. Uma lista é representada como uma sequência de itens na qual cada elemento tem uma posição definida, chamada de índice. **Esses índices começam em zero**, o que significa que o primeiro elemento da lista está localizado na posição zero, o segundo na posição um, e assim por diante. Essa indexação é crucial para acessar, modificar e manipular os elementos contidos na lista.

Uma característica importante das listas é sua **mutabilidade**, ou seja, os dados armazenados podem ser alterados após a criação da lista. Isso inclui a possibilidade de adicionar novos elementos, remover itens existentes, alterar valores em posições específicas e até reorganizar os elementos contidos na estrutura. Essa flexibilidade as torna ideais para situações nas quais os dados precisam ser atualizados ou modificados dinamicamente, como em algoritmos que processam coleções em constante mudança.

As operações mais básicas com listas incluem a adição de elementos, que pode ser feita ao final da lista ou em posições específicas. Da mesma forma, elementos podem ser removidos pelo seu valor ou pela posição em que estão localizados. A substituição de elementos, outra operação comum, permite que valores em posições específicas sejam atualizados diretamente. Além disso, listas podem ser **concatenadas**, o que significa que **duas ou mais listas podem ser combinadas em uma única**, preservando a ordem dos elementos.

No planejamento de uma viagem no tempo, é comum que o viajante queira estabelecer diferentes destinos em ordem cronológica ou simplesmente criar um repertório de anos a serem visitados. Talvez o operador da máquina do tempo esteja interessado em analisar múltiplos períodos históricos ou queira ir e voltar entre diferentes eras para estudar a evolução de sociedades, tecnologias ou eventos específicos. Nesse cenário, lidar com listas torna-se fundamental. Uma lista em Python é uma estrutura dinâmica e flexível, que permite armazenar vários valores em uma única variável, manipular a ordem dos elementos, adicionar ou remover itens e, assim, criar um conjunto ordenado de destinos temporais a serem alcançados.

A seguir é apresentado um código em Python que permite ao usuário criar uma lista de anos desejados. O programa solicitará quantos anos o usuário quer inserir, depois colherá cada ano, armazenando-os em uma lista. Uma vez concluída a entrada, será possível exibir todos os anos armazenados, bem como realizar operações básicas, como a impressão em ordem, adição de um novo ano ou remoção de um destino indesejado. A essência do exercício está em manipular a lista com naturalidade, tornando a experiência do viajante mais prática e intuitiva, como se estivesse preparando uma agenda de viagens comuns, mas agora através do tempo.

```
# Primeiro, solicita-se ao usuário quantos anos ele deseja incluir em sua lista
de destinos temporais.
quantidade = int(input("Quantos anos você deseja adicionar à lista de destinos
temporais? "))
# Cria-se uma lista vazia chamada 'destinos'. Esta lista, inicialmente sem
nenhum elemento,
# servirá como um recipiente para armazenar os anos fornecidos pelo usuário.
destinos = []

# Em seguida, executa-se um loop pelo número de vezes informado na variável
'quantidade'.
# A cada iteração, pede-se ao usuário que insira um ano. Esse valor é convertido
para inteiro,
# pois anos são representados por números inteiros, e em seguida é adicionado ao
final da lista 'destinos'.
for i in range(quantidade):
    ano = int(input("Insira o ano que você deseja visitar: "))
    destinos.append(ano)

# Após o loop, todos os anos solicitados pelo usuário estarão armazenados em
'destinos'.
# Agora, o programa exibe a lista completa, mostrando todos os destinos
temporais escolhidos.
print("Sua lista de destinos temporais:", destinos)

# Para aprofundar a manipulação da lista, podemos ordenar os anos inseridos de
forma crescente.
# Isso pode ser útil se o viajante quiser organizar suas viagens do passado mais
remoto ao futuro mais distante.
destinos.sort()
print("Lista ordenada de destinos temporais:", destinos)

# Suponhamos que o viajante tenha esquecido de inserir um ano importante no
momento da coleta inicial.
# Pode-se inserir facilmente um novo ano usando o método 'append' novamente.
novo_ano = int(input("Insira um ano adicional para adicionar à lista: "))
destinos.append(novo_ano)
print("Lista atualizada após adicionar um novo destino:", destinos)

# Se por acaso o viajante quiser remover um ano incorreto ou indesejado, pode
fazê-lo com o método 'remove'.
# É necessário garantir que o ano exista na lista antes de removê-lo, do
contrário, ocorrerá um erro.
ano_remove = int(input("Insira um ano que deseja remover da lista: "))
if ano_remove in destinos:
    destinos.remove(ano_remove)
    print("Lista após a remoção do destino indesejado:", destinos)
else:
    print("O ano informado não está na lista, nenhuma remoção foi feita.")

# Ao final, o programa exibe novamente a lista para que o viajante verifique se
ela reflete seus planos.
print("Lista final de destinos temporais:", destinos)
```

Figura 88



Lembrete

Nas listas o acesso é feito por meio de índices numéricos que indicam a posição dos elementos; nos dicionários, o acesso ocorre através das chaves, que podem ser de qualquer tipo imutável, como strings, números ou tuplas.

A construção desse código começa perguntando quantos anos o usuário deseja adicionar à lista. Ao usar a função `input()`, o valor obtido é do tipo `string`, e com `int()` convertemos o valor para inteiro, garantindo que quantidade seja um número. Com isso, faremos a execução de um loop `for` na quantidade desejada. Esse loop itera várias vezes, a cada vez pedindo um novo ano, também convertido para inteiro, e inserido na lista `destinos` usando o método `append()`. O uso de `append()` é essencial, pois as listas em Python são dinâmicas: é possível adicionar quantos elementos forem necessários, sem pré-alocar espaço na memória. A figura a seguir mostra o programa em execução:

```
main.py +
1 # Primeiro, solicita-se ao usuário quantos anos ele deseja incluir em sua lista de destinos temporais.
2 quantidade = int(input("Quantos anos você deseja adicionar à lista de destinos temporais? "))
3
4 # Cria-se uma lista vazia chamada 'destinos'. Esta lista, inicialmente sem nenhum elemento,
5 # servirá como um recipiente para armazenar os anos fornecidos pelo usuário.
6 destinos = []
7
Ln: 41, Col: 1
Run Share Command Line Arguments

Quantos anos você deseja adicionar à lista de destinos temporais?
4
Insira o ano que você deseja visitar:
1978
Insira o ano que você deseja visitar:
1500
Insira o ano que você deseja visitar:
1988
Insira o ano que você deseja visitar:
1947
Sua lista de destinos temporais: [1978, 1500, 1988, 1947]
Lista ordenada de destinos temporais: [1500, 1947, 1978, 1988]
Insira um ano adicional para adicionar à lista:
1922
Lista atualizada após adicionar um novo destino: [1500, 1947, 1978, 1988, 1922]
Insira um ano que deseja remover da lista:
1978
Lista após a remoção do destino indesejado: [1500, 1947, 1988, 1922]
Lista final de destinos temporais: [1500, 1947, 1988, 1922]
```

Figura 89 – Lista de destinos temporais

Uma vez finalizada a fase de coleta de dados, a lista completa é exibida. Como a ordem na qual o usuário inseriu os anos pode ser aleatória, o programa demonstra o método `sort()`, que ordena a lista em ordem crescente. A função `print()` é então utilizada para mostrar a lista ordenada. Além

disso, para lidar com possíveis imprevistos, o código permite a inserção tardia de um novo ano e, se necessário, a remoção de um ano indevido. Essas operações ilustram a flexibilidade das listas, que podem ser modificadas a qualquer momento, algo muito importante em um cenário de viagem no tempo, no qual mudanças de plano podem ocorrer conforme o viajante descobre novos interesses ou precisa corrigir erros de planejamento.

Ao término do programa, o código exibe a lista final, permitindo ao viajante avaliar se suas preferências de destinos temporais estão corretas. Caso o viajante note que a lista não reflete fielmente seus planos, ele pode executar novamente o programa, ajustando a quantidade de anos, a ordem deles ou removendo e adicionando outros destinos. Esse ciclo de iteração e refinamento é típico da realidade da programação: **criar, testar, ajustar e executar** novamente.

Outro aspecto das listas é sua capacidade de armazenar tipos de dados heterogêneos. Diferentemente de algumas linguagens de programação, nas quais as listas são restritas a conter elementos de um único tipo, em Python elas podem conter combinações de números, strings, outros objetos e até mesmo outras listas. Essa característica permite que listas sejam usadas para representar estruturas de dados mais complexas, como tabelas ou árvores.

Python fornece uma ampla variedade de métodos para facilitar o trabalho com listas. Esses métodos são operações predefinidas que permitem realizar tarefas comuns de forma eficiente e com uma sintaxe simples. Entre eles estão métodos para adicionar elementos, como `append`, que insere um item no final da lista, e `insert`, que permite a adição em posições específicas. Métodos como `remove` e `pop` são usados para eliminar itens, seja pelo valor, seja pela posição, respectivamente. Além disso, há métodos que ajudam a organizar os elementos, como `sort`, que ordena a lista, e `reverse`, que inverte a ordem dos itens.

As listas também suportam operações que ajudam na análise de dados, como determinar o número de elementos com o método `len` ou contar ocorrências específicas com `count`. Além disso, podem ser iteradas, o que significa que cada elemento pode ser acessado sequencialmente, permitindo que operações sejam realizadas sobre todos os itens de maneira eficiente. Para situações nas quais seja necessário criar cópias das listas, Python oferece maneiras de duplicá-las, tanto em uma versão rasa quanto em versões mais profundas para listas aninhadas.

Além de listar suas próximas paradas temporais, o viajante do tempo quer compreender algumas características dessa lista. Por vezes, precisa saber quantos destinos já foram registrados, determinar qual é o ano mais distante inserido ou até mesmo identificar o ano mais próximo em seu roteiro. Essas informações permitem tomar decisões melhores, seja calculando o esforço energético para viagens muito longas ou reorganizando as prioridades em função de distâncias temporais menores. Ao trabalhar com funções nativas de Python, como `len()`, `max()` e `min()`, é possível obter rapidamente tais estatísticas sobre a lista de destinos escolhidos, sem a necessidade de escrever lógicas elaboradas manualmente.

A seguir, é apresentado um código que coleta uma lista de anos, como no exemplo anterior, e em seguida exibe diversas informações sobre eles, aproveitando essas funções internas do Python. Dessa forma, o programa também oferece uma visão mais profunda do conjunto de destinos definidos.

```
from datetime import datetime

ano_atual = datetime.now().year
quantidade = int(input("Quantos anos de destino você deseja inserir? "))

destinos = []
for i in range(quantidade):
    ano = int(input("Insira um ano que deseja visitar: "))
    destinos.append(ano)

print("Lista de destinos temporais:", destinos)

# A função len() retorna o número total de elementos na lista.
# Isso informa quantos destinos foram registrados, permitindo ao viajante
# avaliar o tamanho de seu itinerário.
total_destinos = len(destinos)
print(f"Você inseriu {total_destinos} destinos na lista.")

# As funções min() e max() permitem identificar o menor e o maior valor na lista.
# Se o viajante planeja visitar anos muito antigos (em relação ao ano atual),
# min() poderá indicar o mais remoto no passado,
# enquanto max() pode mostrar o destino mais distante no futuro.
if total_destinos > 0:
    destino_mais_proximo = min(destinos)
    destino_mais_distante = max(destinos)

    print(f"O destino mais antigo inserido é o ano {destino_mais_proximo}, e o
    mais distante é o ano {destino_mais_distante}.")

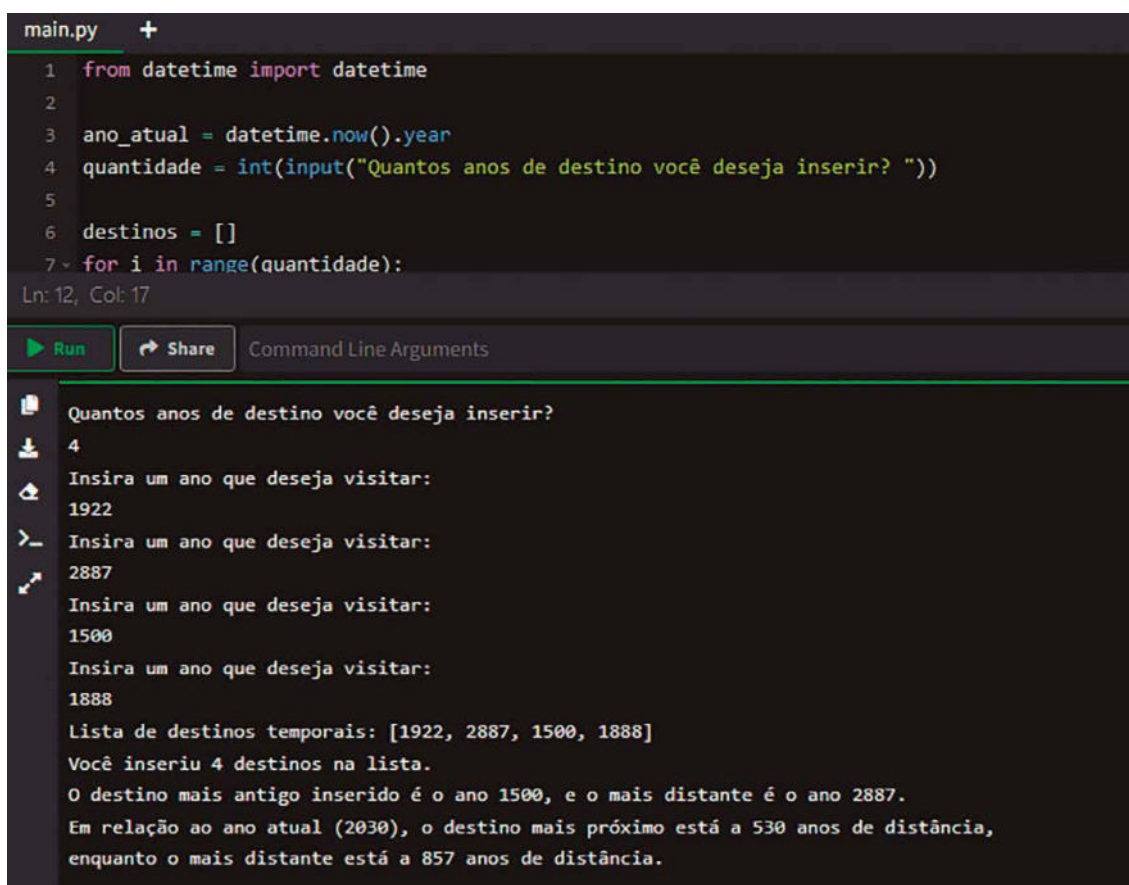
    # É possível calcular quão distantes esses anos estão do presente. Assim, se
    # o viajante tiver colocado anos no passado,
    # a diferença será negativa, mas podemos transformar em valor absoluto para
    # entender a magnitude da viagem.
    diferenca_mais_proximo = abs(destino_mais_proximo - ano_atual)
    diferenca_mais_distante = abs(destino_mais_distante - ano_atual)

    # Ao exibir essas diferenças, o viajante compreende quão longo será o salto
    # temporal até essas épocas.
    print(f"Em relação ao ano atual ({ano_atual}), o destino mais próximo está a
    {diferenca_mais_proximo} anos de distância.")
    print(f"enquanto o mais distante está a {diferenca_mais_distante} anos de
    distância.")
else:
    # Caso o usuário não tenha inserido nenhum destino, não há dados para analisar.
    print("Nenhum destino foi inserido, portanto não há informações adicionais
    a exibir.")
```

Figura 90

Esse código inicia solicitando a quantidade de destinos, em seguida faz um loop e coleta cada ano inserido pelo usuário, inserindo-os na lista destinos. Ao final da coleta, é impressa a lista completa. Com a lista pronta, a chamada a `len(destinos)` fornece o número total de itens, ou seja, quantos anos foram registrados. Isso já responde a um primeiro questionamento: quantos pontos na linha do tempo o viajante deseja visitar.

Em seguida, se a lista tiver pelo menos um elemento, chama-se `min(destinos)` e `max(destinos)`, obtendo o menor e o maior ano entre os inseridos. Essas funções permitem encontrar facilmente o ano mais próximo em termos históricos e o mais distante sem precisar escrever um loop de comparação. A identificação desses extremos pode ajudar a estimar o esforço energético de saltos muito longos ou planejar as viagens em uma sequência coerente.



```
main.py +
1 from datetime import datetime
2
3 ano_atual = datetime.now().year
4 quantidade = int(input("Quantos anos de destino você deseja inserir? "))
5
6 destinos = []
7 for i in range(quantidade):
Ln: 12, Col: 17

Run Share Command Line Arguments

Quantos anos de destino você deseja inserir?
4
Insira um ano que deseja visitar:
1922
Insira um ano que deseja visitar:
2887
Insira um ano que deseja visitar:
1500
Insira um ano que deseja visitar:
1888
Lista de destinos temporais: [1922, 2887, 1500, 1888]
Você inseriu 4 destinos na lista.
O destino mais antigo inserido é o ano 1500, e o mais distante é o ano 2887.
Em relação ao ano atual (2030), o destino mais próximo está a 530 anos de distância,
enquanto o mais distante está a 857 anos de distância.
```

Figura 91 – Lista de destinos temporais com informações adicionais

A seguir, calculam-se as diferenças entre esses anos e o ano atual, obtido pelo `datetime.now().year`. Ao usar `abs(ano - ano_atual)`, transforma-se qualquer valor negativo em positivo, focando na magnitude da distância temporal. Assim, se `destino_mais_proximo` for anterior ao ano atual, a diferença aparecerá como positiva, mostrando quantos anos no passado é preciso viajar; o mesmo vale para destinos no futuro, sempre destacando quantos anos separam o presente do ponto escolhido.

Esse conjunto de operações – contar itens da lista, identificar valores extremos e relacioná-los com o tempo presente – fornece ao viajante uma visão ampliada dos seus destinos temporais, auxiliando-o no planejamento. Ele pode, por exemplo, decidir aumentar a velocidade de viagem, reorganizar a ordem dos destinos ou até mesmo retirar algum destino da lista se o salto for muito extenso. Tudo isso é alcançado com algumas funções nativas do Python, que, aliadas a estruturas de dados adequadas e raciocínio contextual, transformam o simples ato de manipular uma lista de números em uma ferramenta de análise temporal.

Outro ponto interessante é o suporte ao fatiamento (slicing), que permite acessar partes específicas de uma lista sem modificá-la. Isso é feito por meio da especificação de intervalos de índices, permitindo **extrair subconjuntos** de elementos de forma simples e intuitiva. O fatiamento é útil em situações em que é necessário dividir dados em blocos menores para processamento.

A preparação para viagens no tempo pode gerar listas muito longas de anos, especialmente quando o viajante deseja explorar um amplo intervalo histórico, analisando períodos específicos ou pulando diretamente a determinadas épocas. No entanto, muitas vezes não é necessário percorrer todos os destinos contidos na lista. Pode ser interessante selecionar apenas parte da lista: os primeiros anos, os últimos ou até mesmo um intervalo intermediário, analisando apenas uma fração do itinerário.

No código a seguir, o usuário é convidado a inserir uma série de anos que desejaria visitar, criando uma lista extensa de destinos. Uma vez concluída a entrada, o programa demonstra diversas operações de fatiamento, extraindo partes específicas da lista e exibindo os resultados para o viajante. Com isso, o operador da máquina do tempo poderá focar em blocos menores de períodos históricos, talvez analisando apenas os primeiros séculos, um conjunto intermediário de datas ou os anos mais recentes adicionados à lista. Esse tipo de manipulação atende à necessidade de planejar rotas temporais com precisão e escolher fragmentos específicos do tempo para estudo ou exploração.

```
quantidade = int(input("Quantos anos você deseja inserir na lista de destinos  
temporais? "))  
destinos = []  
  
for i in range(quantidade):  
    ano = int(input("Insira um ano: "))  
    destinos.append(ano)  
  
print("Lista completa de destinos:", destinos)  
  
# Abaixo demonstramos o fatiamento de listas.  
# Suponhamos que o viajante deseje olhar apenas os primeiros três destinos  
# inseridos.  
# Usamos destinos[0:3] para obter um sublista começando no índice 0 até, mas não  
# incluindo, o índice 3.  
primeiros_tres = destinos[0:3]  
print("Os primeiros três destinos cadastrados:", primeiros_tres)  
  
# Caso o viajante queira ver apenas os últimos dois destinos,  
# pode-se usar índices negativos. destinos[-2:] retorna a sublista a partir do  
# penúltimo elemento até o final.
```



```
ultimos_dois = destinos[-2:]
print("Os dois últimos destinos inseridos:", ultimos_dois)

# Também é possível pular elementos usando um passo (step).
# Suponha que o viajante queira analisar apenas destinos alternados, saltando um
# elemento a cada passo.
# Usando destinos[::2], obtemos todos os elementos da lista, do início ao fim,
# mas apenas nos índices pares.
alternados = destinos[::2]
print("Destinos alternados (pulando um a cada passo):", alternados)

# Além disso, o viajante pode querer um trecho intermediário da lista.
# Por exemplo, se houver uma longa lista de anos, podemos extrair um pedaço do meio.
# Suponhamos que haja 5 ou mais elementos e queremos pegar do segundo até o
# quarto destino.
# Esse fatiamento é feito com destinos[1:4], retornando elementos nos índices 1,
# 2 e 3.
if len(destinos) >= 5:
    trecho_intermediario = destinos[1:4]
    print("Um trecho intermediário de destinos (índices 1 a 3):", trecho_
intermediario)
else:
    print("A lista não tem destinos suficientes para mostrar um
trecho intermediário.")

# Por fim, podemos inverter a ordem dos destinos usando o fatiamento com
passo negativo.
# destinos[::-1] percorre a lista de trás para frente.
invertidos = destinos[::-1]
print("Destinos em ordem invertida:", invertidos)
```

Figura 92

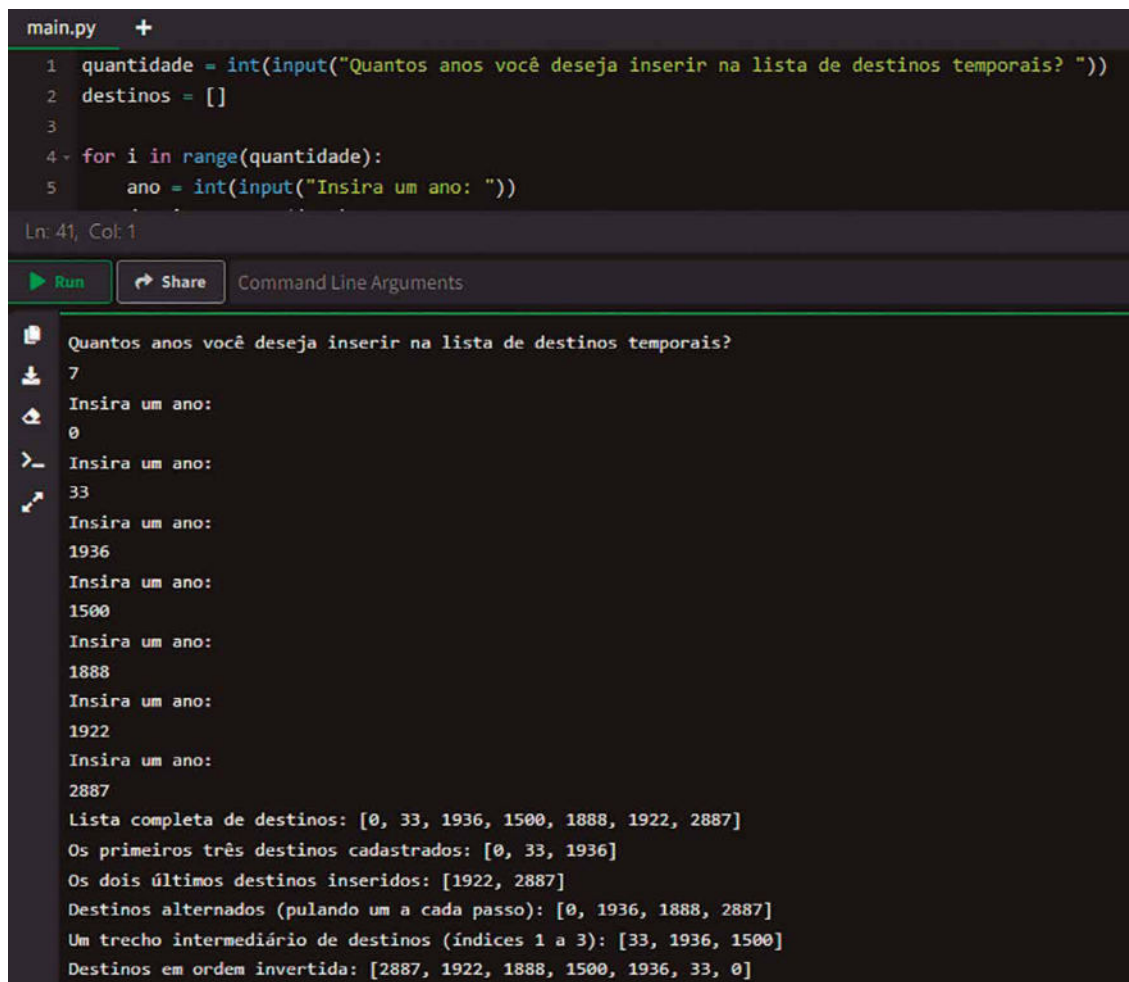
No início do código, solicitamos ao usuário a quantidade de destinos que deseja inserir e, em seguida, um loop coleta cada ano, adicionando-o à lista destinos usando `append()`. Ao final, a lista completa é exibida. A partir daí, o programa demonstra uma série de fatiamentos.

O primeiro exemplo, `primeiros_tres = destinos[0:3]`, acessa os elementos nos índices 0, 1 e 2, produzindo uma sublista dos três primeiros anos. Caso haja mais ou menos destinos, esse fatiamento pode precisar de ajustes, mas o propósito é claro: extrair o início da lista.

Em seguida, `ultimos_dois = destinos[-2:]` mostra como é fácil acessar elementos do final da lista usando índices negativos. O `-2` significa o penúltimo elemento, e a ausência de valor após os dois-pontos significa continuar até o final da lista; é uma forma simples de ver as últimas inserções feitas pelo viajante.

O exemplo `alternados = destinos[::2]` ilustra o uso do parâmetro de passo (step) no fatiamento. Ao deixar o início e o fim vazios, mas especificar `::2`, percorremos a lista do começo ao fim, porém selecionando apenas elementos em índices pares, pulando um a cada passo. Isso permite filtrar a lista de maneira interessante, talvez ignorando anos que não sejam tão relevantes ou analisando apenas uma amostra esparsa de destinos.

Para o trecho intermediário, `destinos[1:4]` extrai os elementos nos índices 1, 2 e 3, desde que a lista seja grande o suficiente. Essa operação ajuda a focar em um subconjunto da linha do tempo, possivelmente analisando apenas um período de três anos contíguos no meio da lista, permitindo recortar a história em pedaços manejáveis. A figura a seguir apresenta um exemplo da execução do programa:



```
main.py +
1 quantidade = int(input("Quantos anos você deseja inserir na lista de destinos temporais? "))
2 destinos = []
3
4 for i in range(quantidade):
5     ano = int(input("Insira um ano: "))
6
Ln: 41, Col: 1
Run Share Command Line Arguments
Quantos anos você deseja inserir na lista de destinos temporais?
7
Insira um ano:
0
Insira um ano:
33
Insira um ano:
1936
Insira um ano:
1500
Insira um ano:
1888
Insira um ano:
1922
Insira um ano:
2887
Lista completa de destinos: [0, 33, 1936, 1500, 1888, 1922, 2887]
Os primeiros três destinos cadastrados: [0, 33, 1936]
Os dois últimos destinos inseridos: [1922, 2887]
Destinos alternados (pulando um a cada passo): [0, 1936, 1888, 2887]
Um trecho intermediário de destinos (índices 1 a 3): [33, 1936, 1500]
Destinos em ordem invertida: [2887, 1922, 1888, 1500, 1936, 33, 0]
```

Figura 93 – Fatiando a lista de destinos de diversos modos

Por fim, note que `destinos[::-1]` usa um passo negativo, invertendo a ordem de todos os destinos. Isso pode ser útil se o viajante mudar de ideia sobre o sentido de sua exploração temporal, desejando analisar ou visitar as épocas na ordem oposta àquela originalmente planejada.

Em Python, tuplas e listas são estruturas de dados usadas para armazenar coleções de elementos. Ambas permitem agrupar itens de diferentes tipos e acessá-los por índice, mas diferem em várias características importantes. A principal diferença entre tuplas e listas está na mutabilidade.

As listas são mutáveis, permitindo alterações nos elementos, inserções, remoções e reordenações após a criação. Essa diferença torna as tuplas úteis para representar coleções de dados que não devem ser modificadas, garantindo segurança e integridade, enquanto as listas são mais adequadas para situações em que mudanças dinâmicas nos dados são necessárias.

Além disso, há implicações no desempenho e no uso da memória. Devido à sua imutabilidade, tuplas são mais leves e, em geral, ocupam menos memória do que listas. Isso também as torna ligeiramente mais rápidas para certas operações, como iteração. Por outro lado, a mutabilidade das listas permite maior flexibilidade em termos de manipulação de dados, como redimensionamento e reordenação.

As listas em Python desempenham um papel central em aplicações práticas de IA e ciência de dados. Por serem uma estrutura de dados flexível, eficiente e amplamente suportada, elas são frequentemente usadas em várias etapas do pipeline de desenvolvimento e análise, desde a manipulação inicial de dados até a implementação de algoritmos básicos. Embora bibliotecas especializadas como NumPy e Pandas sejam comumente preferidas em cenários avançados devido ao desempenho otimizado e funcionalidades adicionais, as listas ainda são fundamentais para muitos casos, especialmente quando a simplicidade e a flexibilidade são prioridades.

No contexto de ciência de dados, as listas são usualmente utilizadas para armazenar coleções de dados heterogêneos ou estruturados de maneira simples. Por exemplo, listas podem conter registros de informações, como as características de um conjunto de observações ou medições. Durante a etapa de pré-processamento de dados, listas são úteis para tarefas como limpar, organizar ou reformatar os dados antes de serem analisados ou inseridos em modelos. Isso inclui transformar strings em números, remover valores ausentes ou dividir conjuntos de dados em subconjuntos específicos para análise.

Em IA, listas são úteis para implementar representações básicas de dados, como matrizes de entrada ou saídas de um modelo. Elas ainda são amplamente usadas para organizar dados em situações nas quais a estrutura não é fixa, como listas de documentos para processamento de linguagem natural ou grafos dinâmicos para problemas de redes neurais recorrentes. Além disso, listas podem ser usadas para representar sequências de estados em algoritmos de aprendizado por reforço ou armazenar resultados intermediários durante a execução de pipelines.

As listas desempenham um papel crucial na prototipagem de algoritmos. Em muitas aplicações de IA, como classificação, regressão e clustering, listas são usadas para manter as características de entrada, as previsões de modelos e as respostas esperadas. Essa simplicidade permite que os pesquisadores e desenvolvedores testem rapidamente ideias sem a complexidade de configurações mais elaboradas. Além disso, listas permitem representar conjuntos de exemplos e suas etiquetas em aprendizado supervisionado de forma simples e intuitiva.

Outra aplicação relevante é na simulação de dados e na visualização inicial. Por exemplo, listas são frequentemente usadas para armazenar os resultados de simulações, que podem ser posteriormente analisados ou plotados em gráficos. Elas também são úteis para armazenar as saídas de experimentos em inteligência artificial, como os valores de perda e precisão ao longo das épocas de treinamento, ajudando os desenvolvedores a monitorar o desempenho de seus modelos.

5.2 Dicionários: criação e uso

Imagine que você está tentando organizar informações para facilitar sua busca, como associar o nome de uma pessoa ao seu número de telefone. Uma lista poderia ser usada, mas ela funciona como uma folha de papel longa e desorganizada, na qual você precisaria procurar linha por linha até encontrar o dado desejado. Esse método, embora funcional, rapidamente se torna ineficiente à medida que a quantidade de informações cresce. Para resolver esse problema, entra em cena o dicionário, que funciona como uma agenda telefônica bem organizada. Em vez de buscar manualmente entre os itens, cada nome (a chave) está diretamente ligado ao número correspondente (o valor), permitindo localizar rapidamente o que você precisa.

Um dicionário em Python é uma estrutura de dados que armazena elementos em pares de chave-valor. As chaves funcionam como identificadores únicos, enquanto os valores contêm as informações associadas a essas chaves. Por exemplo, uma chave pode ser o nome de um cliente, e o valor pode ser seu endereço ou histórico de compras. Essa relação direta entre chave e valor é a essência dos dicionários, permitindo que os dados sejam acessados de forma eficiente sem depender de posições ou índices fixos, como em uma lista. Isso faz com que dicionários sejam especialmente úteis em aplicações que exigem uma rápida recuperação de informações.

A principal diferença entre um dicionário e uma lista é a forma como os dados são organizados e acessados. Enquanto uma lista armazena itens em uma sequência ordenada, acessados por índices numéricos (0, 1, 2 e assim por diante), um dicionário não depende dessa ordem. Em vez disso, as chaves são usadas para acessar os valores. Essa característica confere aos dicionários uma flexibilidade maior, permitindo que as chaves sejam de qualquer tipo imutável, como strings, números ou até mesmo tuplas. Como resultado, eles são ideais para representar relacionamentos entre dados ou coleções nos quais cada elemento tem uma identidade única.

Além disso, dicionários são extremamente eficientes em termos de busca e manipulação de dados. Quando você sabe a chave, pode acessar o valor associado diretamente, sem precisar iterar por toda a estrutura, como seria necessário em uma lista. Isso os torna indispensáveis em uma ampla gama de aplicações, desde armazenamento de configurações de programas até representação de dados mais complexos, como registros de usuários ou tabelas de dados organizadas por atributos. Por essas razões, os dicionários são uma das ferramentas mais poderosas e versáteis disponíveis no Python, essenciais para lidar com dados estruturados de maneira clara e eficiente.

A estrutura chave-valor, que é a essência dos dicionários em Python, é também o núcleo do **JSON** (JavaScript Object Notation), um dos formatos mais populares para a troca de informações entre sistemas remotos. JSON é um padrão leve, legível tanto por humanos quanto por máquinas, e sua simplicidade o tornou uma escolha predominante em APIs (Application Programming Interfaces), serviços web e integrações entre sistemas. Ele utiliza exatamente o mesmo conceito de pares chave-valor para organizar os dados, o que permite descrever de forma estruturada e clara informações que precisam ser enviadas ou recebidas.

No JSON, as chaves são sempre strings, enquanto os valores podem ser de diferentes tipos básicos, como números, strings, booleanos, listas (representadas como arrays) ou até mesmo outros objetos JSON, criando estruturas aninhadas. Essa flexibilidade permite que o JSON represente desde dados simples, como uma lista de produtos com preços, até dados complexos, como configurações de software ou respostas detalhadas de um serviço web. A sintaxe do JSON é amplamente suportada por várias linguagens de programação, incluindo Python, o que facilita sua adoção em diferentes contextos.

Por ser diretamente inspirado pela notação de objetos em JavaScript, o JSON ganhou rapidamente popularidade como um formato padrão em sistemas baseados na web. Ele substituiu o padrão XML (eXtensible Markup Language) em muitas aplicações por ser mais compacto e fácil de manipular. Em Python, a manipulação de JSON é extremamente simples devido à similaridade entre o formato JSON e os dicionários. Python fornece o módulo `json`, que permite converter facilmente dicionários em strings JSON para envio ou armazenamento, e strings JSON de volta em dicionários para processamento interno. Essa integração perfeita torna o Python uma linguagem ideal para lidar com sistemas que dependem de JSON.

Além disso, o JSON não se limita à troca de dados. Ele também é usado como formato de armazenamento em bancos de dados não relacionais, como o MongoDB, e como configuração em diversos softwares modernos. Essa ubiquidade reflete a eficiência e a praticidade de uma estrutura baseada em chave-valor para representar dados estruturados de maneira clara, legível e adaptável a diferentes contextos. A combinação dessa simplicidade com a interoperabilidade garantiu ao JSON um lugar central no ecossistema de desenvolvimento de sistemas modernos.



Saiba mais

O livro acentuado a seguir aborda diversos aspectos do Python, incluindo o tratamento de dados com JSON, oferecendo uma compreensão profunda da linguagem e suas aplicações práticas:

RAMALHO, L. *Python fluente: programação clara, concisa e eficaz*. São Paulo: Novatec, 2015.

A segunda indicação é um livro focado em iniciantes e que ensina como utilizar Python para automatizar tarefas, incluindo a manipulação de arquivos JSON, proporcionando uma base sólida para o uso prático da linguagem:

SWEIGART, A. *Automatize tarefas maçantes com Python: programação prática para verdadeiros iniciantes*. São Paulo: Novatec, 2015.

Ao preparar uma jornada no tempo, não é apenas a tecnologia da máquina ou o destino final que importam. O perfil do viajante em si pode ser um fator determinante para o sucesso da empreitada. Há situações em que o ano de origem do viajante, sua idade e até mesmo seu nome podem exercer influência

na calibragem dos instrumentos, na escolha do período a ser visitado e na credibilidade que ele terá ao interagir com diferentes povos e culturas. Ao estruturar essas informações, é útil armazená-las de forma organizada, permitindo consultá-las com facilidade a qualquer momento. O dicionário em Python é a estrutura de dados perfeita para essa tarefa, pois funciona como uma coleção de pares chave-valor, possibilitando acessar e modificar dados pelo nome da chave, em vez de depender de índices numéricos.

O código a seguir cria um dicionário chamado `perfil_viajante`, que conterá informações básicas sobre o usuário: seu nome, sua idade e o ano de origem. Esses dados são solicitados ao próprio viajante, simulando o momento em que ele registra seus dados na máquina do tempo antes de partir. Uma vez armazenadas, as informações podem ser exibidas, manipuladas ou enriquecidas com novos dados, conferindo maior contexto e dando vida ao personagem que efetivamente realizará o salto temporal.

```
# Inicialmente, cria-se um dicionário vazio para armazenar as informações
do viajante.
perfil_viajante = {}

# A função input() é usada para coletar o nome do usuário.
# O nome será uma string, representando a identidade do viajante.
nome = input("Insira o seu nome: ")

# Agora, pede-se a idade, que é convertida para inteiro, já que a idade é
representada por um valor numérico.
idade = int(input("Insira a sua idade: "))

# Por fim, solicita-se o ano de origem do viajante.
# Esse ano será igualmente transformado em inteiro, facilitando cálculos
futuros, caso seja necessário.
ano_origem = int(input("Insira o ano de origem: "))

# Com as informações coletadas, insere-se cada uma no dicionário 'perfil_
viajante' através de pares chave-valor.
# Aqui, 'nome', 'idade' e 'ano_origem' são as chaves, enquanto as variáveis que
contêm os dados do usuário são os valores.
perfil_viajante["nome"] = nome
perfil_viajante["idade"] = idade
perfil_viajante["ano_origem"] = ano_origem

# Agora que o dicionário está completo, é possível exibir as
informações armazenadas.
# Ao imprimir o dicionário inteiro, vê-se uma representação clara dos dados,
permitindo verificar se tudo foi inserido corretamente.
print("Perfil do viajante:", perfil_viajante)

# Se o operador da máquina do tempo desejar acessar um dado específico, basta
usar a chave correspondente.
# Por exemplo, para imprimir apenas o nome do viajante, pode-se fazer:
print("Nome do viajante:", perfil_viajante["nome"])

# Pode-se ainda imaginar que, dependendo do ano de origem, certos ajustes
precisem ser feitos.
# Ao obter o ano de origem, seria possível realizar cálculos ou comparações com
o ano atual, preparando assim o ambiente para a viagem.
# Este exemplo ilustra apenas a leitura dos dados, mas o dicionário poderia ser
atualizado posteriormente,
# caso fosse necessário inserir mais informações, como a nacionalidade do
viajante, sua profissão ou suas habilidades linguísticas.
```

Figura 94



Lembrete

A principal diferença entre tuplas e listas está na mutabilidade.

No código apresentado, o dicionário `perfil_viajante` inicia vazio e é gradualmente preenchido com dados coletados diretamente do usuário. Cada par chave-valor é uma associação lógica: `nome` aponta para uma string que identifica a pessoa, `idade` para um inteiro representando quantos anos o viajante tem no momento da partida e `ano_origem` para outro inteiro que indica de que época histórica essa pessoa está partindo. Essa flexibilidade facilita a manutenção e a evolução do programa. Se novas informações forem necessárias, basta acrescentar novas chaves ao dicionário, tornando o perfil do viajante cada vez mais rico. A figura a seguir ilustra a execução do programa:

```
main.py +
1 # Inicialmente, cria-se um dicionário vazio para armazenar as informações do viajante.
2 perfil_viajante = {}
3
4 # A função input() é usada para coletar o nome do usuário.
5 # O nome será uma string, representando a identidade do viajante.
6 nome = input("Insira o seu nome: ")
7
8 # Agora, pede-se a idade, que é convertida para inteiro, já que a idade é representada por um valor numérico.
9 idade = int(input("Insira a sua idade: "))

Ln 33, Col: 1
Run Share Command Line Arguments

Insira o seu nome:
Biff Tannen
Insira a sua idade:
28
Insira o ano de origem:
1990
Perfil do viajante: {'nome': 'Biff Tannen', 'idade': 28, 'ano_origem': 1990}
Nome do viajante: Biff Tannen
```

Figura 95 – Uso de dicionário para armazenar as informações do viajante

Dessa forma, o uso de um dicionário permite organizar dados heterogêneos de maneira clara e acessível. O viajante não precisa mais se lembrar manualmente de seu ano de origem ou de sua idade no momento da viagem, pois essas informações estão todas concentradas em um só lugar.

A jornada pelo tempo não se resume apenas a decidir um ano de destino. Muitas vezes, o viajante quer ter contexto sobre o período histórico que está prestes a visitar. Ter dados sobre eventos importantes, personagens marcantes ou circunstâncias culturais pode guiar as ações do explorador temporal, ajudando-o a compreender o que encontrará ao chegar naquele momento específico. Utilizar dicionários aninhados em Python é uma forma eficiente de estruturar essas informações. Com **dicionários aninhados**, é possível ter um dicionário principal em que cada chave é um ano, e cada valor é outro dicionário repleto de detalhes sobre o evento que ocorreu naquele ano. Dessa maneira, o programa adquire a capacidade de organizar e apresentar contexto histórico de forma hierárquica, clara e expansível.

O código a seguir cria um dicionário `eventos_historicos` no qual o usuário pode inserir diferentes anos e, para cada ano, um conjunto de detalhes, como o nome do evento, a localização e a relevância. Cada chave do dicionário principal será um ano, e o valor será outro dicionário contendo esses pormenores. Ao final, o programa mostra como acessar essas informações, oferecer um panorama histórico enriquecido e, inclusive, permitir que o usuário acrescente novos eventos a qualquer momento.

```
# Cria um dicionário vazio para armazenar os eventos históricos detalhados.
eventos_historicos = {}

# Solicita ao usuário quantos eventos deseja registrar.
quantidade = int(input("Quantos eventos históricos você deseja adicionar? "))

# Para cada evento, o usuário insere um ano e os detalhes do evento, que serão
# armazenados em um dicionário interno.
for i in range(quantidade):
    ano = int(input("\nInsira o ano do evento histórico: "))
    nome_evento = input("Insira o nome/título do evento: ")
    localizacao = input("Insira a localização do evento: ")
    relevancia = input("Insira a relevância do evento (por exemplo, 'marco
político', 'descoberta científica', etc.): ")

    # Cria um dicionário interno contendo os detalhes do evento para o ano
    # específico.
    detalhes = {
        "nome_evento": nome_evento,
        "localizacao": localizacao,
        "relevancia": relevancia
    }

    # Associa o ano ao dicionário interno no dicionário principal.
    eventos_historicos[ano] = detalhes

# Exibe todos os eventos históricos registrados, mostrando o dicionário
# completo.
print("\nEventos Históricos Registrados:")
print(eventos_historicos)

# Caso o usuário deseje acessar informações sobre um ano específico, pode fazê-lo
# utilizando a chave do dicionário.
ano_consulta = int(input("\nInsira um ano para consultar os detalhes do evento:
"))
if ano_consulta in eventos_historicos:
    evento = eventos_historicos[ano_consulta]
    print(f"\nDetalhes do evento no ano {ano_consulta}:")
    print(f"Nome do evento: {evento['nome_evento']}")
    print(f"Localização: {evento['localizacao']}")
    print(f"Relevância: {evento['relevancia']}")
else:
    print("Não há registro de evento para o ano informado.")
# Aqui, o viajante do tempo pode imaginar o quão valioso é possuir tais
# informações em um dicionário aninhado.
# Ao usar essa estrutura, é possível adicionar novos anos facilmente, alterar
# detalhes, remover ou atualizar eventos,
# tudo com acesso direto por ano. Além disso, a hierarquia deixa clara a relação
# entre o ano (chave externa)
# e seus detalhes (dicionário interno).
```

Figura 96

Novamente o dicionário principal, `eventos_historicos`, começa vazio e, a cada iteração do loop, recebe um novo par chave-valor, no qual a chave é um ano inteiro e o valor é outro dicionário contendo detalhes sobre o evento. Esse segundo dicionário interno – o dicionário aninhado – armazena informações como nome do evento, localização e relevância, todas associadas a chaves descritivas. Assim, o conjunto resultante é uma estrutura hierarquizada e coerente: primeiro o ano, depois os detalhes.

Ao imprimir `eventos_historicos`, vemos um dicionário cujas chaves são anos e cujos valores são outros dicionários. Isso facilita a compreensão e a manipulação do contexto histórico. Caso seja necessário, o programa pode ser facilmente estendido para incluir novas chaves no dicionário interno, como "participantes", "duração" ou "impacto cultural". Da mesma forma, para obter as informações de um ano específico, basta acessar `eventos_historicos[ano_desejado]`, obtendo o dicionário interno e, em seguida, acessar as chaves individuais para mostrar detalhes.

A seguir ilustraremos a execução do código. Em negrito estão as informações que foram sendo digitadas ao longo da execução.

Quantos eventos históricos você deseja adicionar?

3

Insira o ano do evento histórico:

1500

Insira o nome/título do evento:

Descobrimento do Brasil

Insira a localização do evento:

Porto Seguro, Brasil

Insira a relevância do evento (por exemplo, 'marco político', 'descoberta científica', etc.):

Marco geográfico e cultural

Insira o ano do evento histórico:

1822

Insira o nome/título do evento:

Independência do Brasil

Insira a localização do evento:

Rio de Janeiro, Brasil

Insira a relevância do evento (por exemplo, 'marco político', 'descoberta científica', etc.):

Marco político

Insira o ano do evento histórico:

1888

Insira o nome/título do evento:

Abolição da Escravidão

Insira a localização do evento:

Brasil

Insira a relevância do evento (por exemplo, 'marco político', 'descoberta científica', etc.):

Marco social

Eventos Históricos Registrados:

```
{1500: {'nome_evento': 'Descobrimento do Brasil', 'localizacao': 'Porto Seguro, Brasil', 'relevancia': 'Marco geográfico e cultural'}, 1822: {'nome_evento': 'Abolição da Escravidão', 'localizacao': 'Brasil', 'relevancia': 'Marco social'}}
```

Insira um ano para consultar os detalhes do evento:

1500

Detalhes do evento no ano 1500:

Nome do evento: Descobrimento do Brasil

Localização: Porto Seguro, Brasil

Relevância: Marco geográfico e cultural

** Process exited - Return Code: 0 **

Press Enter to exit terminal

Figura 97

Suponha agora que o viajante deseja estabelecer compromissos e tarefas a cumprir quando chegar a essas épocas. Talvez o objetivo seja assistir à coroação de um monarca, presenciar a descoberta de um novo continente, negociar com antigos comerciantes ou até mesmo participar de um evento cultural milenar. Para gerenciar esses compromissos, é possível usar dicionários em Python, nos quais cada ano serve como chave, e o valor pode ser uma lista ou outra estrutura para armazenar diversos compromissos. Desse modo, a "agenda temporal" do viajante fica clara e flexível, permitindo adicionar, remover ou consultar tarefas conforme necessário.

No código a seguir, o usuário pode inserir compromissos associados a diferentes anos, armazenando esses dados em um dicionário chamado `agenda_temporal`. Caso o ano já exista, novos compromissos são acrescentados; se o ano não existir, ele é criado. O viajante também poderá consultar os compromissos de um ano específico, verificando o que o espera naquela época, seja um único evento isolado, seja uma lista extensa de afazeres históricos.

```
# Cria um dicionário vazio para armazenar a agenda temporal.
# Cada chave será um ano (inteiro) e cada valor será uma lista de compromissos (strings).
agenda_temporal = {}

# Solicita ao usuário quantos compromissos deseja registrar inicialmente.
quantidade = int(input("Quantos compromissos temporais você deseja adicionar? "))

# Para cada compromisso, o usuário insere um ano e a descrição do compromisso.
for i in range(quantidade):
    ano = int(input("\nInsira o ano do compromisso: "))
    compromisso = input("Insira a descrição do compromisso: ")

    # Verifica se o ano já existe na agenda.
    # Caso exista, apenas adiciona o novo compromisso à lista.
    # Caso contrário, cria uma nova entrada para o ano com esse compromisso.
    if ano in agenda_temporal:
        agenda_temporal[ano].append(compromisso)
```

```
else:
    agenda_temporal[ano] = [compromisso]

# Após adicionar todos os compromissos, exibe a agenda completa.
print("\nAgenda Temporal Completa:")
print(agenda_temporal)

# Agora, oferece-se a opção de consultar os compromissos de um ano específico.
ano_consulta = int(input("\nInsira um ano para consultar seus compromissos: "))
if ano_consulta in agenda_temporal:
    # Caso o ano exista, exibe os compromissos associados a ele.
    print(f"Compromissos para o ano {ano_consulta}:")
    for idx, compromisso in
enumerate(agenda_temporal[ano_consulta], start=1):
        print(f"{idx}. {compromisso}")
else:
    # Caso o ano não exista na agenda, informa que não há compromissos cadastrados.
    print(f"Não há compromissos cadastrados para o ano {ano_consulta}.")

# O viajante pode ainda desejar atualizar ou adicionar mais compromissos mesmo
após a primeira inserção.
# Por exemplo, pode-se criar um loop para permitir que o usuário continue
interagindo com a agenda,
# inserindo novos compromissos ou consultando até decidir encerrar.
opcao = input("\nDeseja adicionar mais compromissos? (s/n) ")
while opcao.lower() == 's':
    ano_adicionar = int(input("Insira o ano do novo compromisso: "))
    novo_compromisso = input("Insira a descrição do novo compromisso: ")

    if ano_adicionar in agenda_temporal:
        agenda_temporal[ano_adicionar].append(novo_compromisso)
    else:
        agenda_temporal[ano_adicionar] = [novo_compromisso]
    print("Compromisso adicionado com sucesso.")
    opcao = input("Deseja adicionar mais compromissos? (s/n) ")

# Ao final, a agenda atualizada é exibida novamente, refletindo todas as
mudanças feitas.
print("\nAgenda Temporal Atualizada:")
print(agenda_temporal)
```

Figura 98

No início do código, o usuário determina quantos compromissos deseja inserir. Cada compromisso é associado a um ano. Se esse ano já existir como chave no dicionário `agenda_temporal`, o compromisso é simplesmente anexado à lista de compromissos daquele ano. Se não existir, cria-se uma nova chave para esse ano, com o valor sendo uma lista contendo o primeiro compromisso. Ao terminar essa fase, o dicionário `agenda_temporal` já reflete um conjunto inicial de compromissos distribuídos ao longo de diferentes anos.

O trecho `for idx, compromisso in enumerate(agenda_temporal[ano_consulta], start=1):` desempenha a tarefa de iterar sobre os compromissos associados a um

ano específico, previamente selecionado pelo usuário (armazenado em `ano_consulta`), e exibi-los numerados de forma sequencial. Para entender em detalhes, é importante observar os seguintes componentes:

- `agenda_temporal[ano_consulta]`: essa expressão acessa a lista de compromissos associada ao ano fornecido pelo usuário. A variável `agenda_temporal` é um dicionário no qual cada chave é um ano e o valor correspondente é uma lista de compromissos. Ao buscar por `agenda_temporal[ano_consulta]`, o programa recupera a lista específica para o ano solicitado.
- `enumerate`: a função `enumerate` é uma ferramenta poderosa em Python que permite iterar sobre uma sequência (neste caso, uma lista) enquanto mantém uma contagem do número de iterações. O `enumerate` retorna pares de valores, no qual o primeiro elemento do par é o índice da iteração (começando por padrão em zero, mas aqui configurado para iniciar em 1) e o segundo é o elemento atual da sequência.
- `idx, compromisso`: no contexto do loop, `enumerate` produz dois valores para cada iteração:
 - `idx`: o índice numérico, que aqui começa em 1 por causa do argumento `start=1`. Esse índice é usado para numerar os compromissos de forma intuitiva ao exibi-los.
 - `compromisso`: o elemento atual da lista de compromissos, ou seja, a descrição textual de cada compromisso.
- **Iteração sobre a lista**: o loop percorre todos os elementos da lista `agenda_temporal[ano_consulta]`, com cada elemento sendo tratado como compromisso. O índice `idx` incrementa automaticamente a cada iteração, garantindo que cada compromisso seja numerado corretamente.
- **Objetivo do loop**: o propósito desse trecho é exibir cada compromisso relacionado ao ano consultado de maneira organizada e numerada, o que é especialmente útil para o usuário entender quais compromissos estão cadastrados. Durante cada iteração, o programa pode usar `idx` e `compromisso` para formatar e exibir as informações.

Vamos dar um exemplo prático do que acabamos de detalhar. Suponha que `agenda_temporal` contenha:

```
{
    2025: ["Conferência sobre IA", "Lançamento do novo satélite"],
    2030: ["Reunião intergaláctica", "Teste do protótipo de viagem no tempo"]
}
```

Figura 99

Agora suponha que o usuário insira 2030 para consulta. A expressão `agenda_temporal[2030]` acessará a lista:

```
["Reunião intergaláctica", "Teste do protótipo de viagem no tempo"]
```

Figura 100

O loop `for idx, compromisso in enumerate(agenda_temporal[ano_consulta], start=1)` produzirá:

- **Na primeira iteração:** `idx = 1`, `compromisso = "Reunião intergaláctica"`.
- **Na segunda iteração:** `idx = 2`, `compromisso = "Teste do protótipo de viagem no tempo"`.

O resultado exibido será:

1. Reunião intergaláctica
2. Teste do protótipo de viagem no tempo

Figura 101

Esse uso do `enumerate` torna o código mais legível e eficiente. Ele combina a funcionalidade de percorrer uma lista com a geração automática de índices numéricos, eliminando a necessidade de gerenciar manualmente contadores. Além disso, o `start=1` melhora a experiência do usuário, exibindo números que começam em 1, algo mais natural para visualização, em vez do padrão zero usado internamente pelo Python.

O código permite consultar os compromissos de um ano específico, iterando sobre a lista de compromissos armazenada naquela chave. Cada compromisso é exibido acompanhado de um índice, facilitando a leitura. Caso o ano consultado não exista no dicionário, informa-se que não há compromissos para aquele ano.

Por fim, o usuário tem a opção de continuar adicionando compromissos, aprofundando a flexibilidade do sistema. A qualquer momento, novos anos podem ser criados e novos compromissos inseridos em anos já existentes, tornando essa agenda realmente dinâmica. Essa abordagem oferece ao viajante um controle fino sobre seus compromissos temporais: ele pode planejar uma visita rápida a um ano distante para participar de uma coroação, inserir compromissos mais longos que exijam múltiplas atividades no mesmo ano ou até mesmo construir um plano complexo de eventos históricos a serem acompanhados.

A variável `opcao` contém a resposta do usuário, que é capturada através de um comando `input` em uma etapa anterior do código. O método `.lower()` transforma qualquer string em letras minúsculas, garantindo que a comparação seja feita de forma insensível a maiúsculas ou minúsculas. Por exemplo, tanto 'S' quanto 's' serão convertidos para 's', evitando problemas causados por diferentes estilos de digitação.

Esse loop `while` é usado para manter o programa em um estado interativo, permitindo que o usuário adicione quantos compromissos desejar. Sempre que o programa pergunta Deseja adicionar mais compromissos? (s/n) e o usuário insere 's' (ou 'S'), o loop continua e o programa solicita as informações de um novo compromisso. Quando o usuário digita algo diferente de 's', como 'n', o loop é encerrado, e o programa prossegue com a próxima etapa, que no caso é exibir a agenda atualizada.

A seguir ilustraremos a execução do código. Em negrito, as informações que foram sendo digitadas ao longo da execução.

```
Quantos compromissos temporais você deseja adicionar?
```

```
3
```

```
Insira o ano do compromisso:
```

```
1500
```

```
Insira a descrição do compromisso:
```

```
Assistir à missa de fundação do Brasil
```

```
Insira o ano do compromisso:
```

```
1822
```

```
Insira a descrição do compromisso:
```

```
Ouvir, às margens do Ipiranga, o grito da independência do Brasil
```

```
Insira o ano do compromisso:
```

```
1922
```

```
Insira a descrição do compromisso:
```

```
Prestigiar a semana de arte moderna
```

```
Agenda Temporal Completa:
```

```
{1500: ['Assistir a missa de fundação do Brasil'], 1822: ['Ouvir, às margens do  
Ipiranga, o grito da independência do Brasil'], 1922: ['Prestigiar a semana de  
arte moderna']}
```

```
Insira um ano para consultar seus compromissos:
```

```
1822
```

```
Compromissos para o ano 1822:
```

```
1. Ouvir, às margens do Ipiranga, o grito da independência do Brasil
```

```
Deseja adicionar mais compromissos? (s/n)
```

```
n
```

```
Agenda Temporal Atualizada:
```

```
{1500: ['Assistir a missa de fundação do Brasil'], 1822: ['Ouvir, às margens do  
Ipiranga, o grito da independência do Brasil'], 1922: ['Prestigiar a semana de  
arte moderna']}
```

```
** Process exited - Return Code: 0 **
```

```
Press Enter to exit terminal
```

Figura 102

O Python, em teoria, pode armazenar uma quantidade muito grande de itens em um dicionário, limitada principalmente pela memória disponível no sistema no qual o programa está sendo executado. Não há um limite explícito imposto pela linguagem para o número de itens que um dicionário pode conter. No entanto, na prática, alguns fatores influenciam a capacidade de um dicionário de armazenar grandes quantidades de dados:

- **Limitação pela memória do sistema:** cada entrada em um dicionário ocupa espaço na memória, incluindo a chave, o valor e as estruturas internas do próprio dicionário para gerenciar os pares chave-valor. Se o dicionário crescer muito, ele pode eventualmente consumir toda a memória disponível, o que causará um erro de falta de memória (`MemoryError`).
- **Arquitetura do Python:** a implementação do Python e o sistema subjacente (32 bits ou 64 bits) influenciam o uso de memória. Em sistemas de 64 bits, a quantidade máxima de memória que pode ser endereçada é muito maior, permitindo dicionários maiores.
- **Eficiência do dicionário:** dicionários em Python são implementados como tabelas hash, que gerenciam automaticamente o crescimento da estrutura interna à medida que novos itens são adicionados. Dessa forma, as operações em dicionários (inserção, busca, remoção) permanecem eficientes, mesmo com um grande número de itens. No entanto, cada redimensionamento consome memória e tempo de processamento, o que pode impactar o desempenho em casos extremos.
- **Tamanho de chaves e valores:** isso também afeta o consumo total de memória. Dicionários com chaves e valores pequenos (como inteiros ou strings curtas) consomem menos memória por item em comparação com dicionários que armazenam objetos grandes ou estruturas de dados complexas.



Observação

Embora não haja um limite teórico, na prática, um dicionário Python pode facilmente conter **milhões de itens** em sistemas modernos com memória suficiente. Por exemplo, em testes realizados com máquinas de 64 bits e dezenas de gigabytes de memória RAM, é comum armazenar dicionários com dezenas ou centenas de milhões de entradas, desde que haja memória disponível.

Ao preparar uma viagem no tempo, o viajante nem sempre pode se expressar livremente utilizando a linguagem e as expressões contemporâneas de seu ponto de partida temporal. É comum que, ao chegar a uma época distante, as palavras, gírias ou construções frasais tenham um sentido diferente ou até mesmo não existam. Garantir que o viajante possa comunicar-se adequadamente com as pessoas daquela era é fundamental para evitar mal-entendidos, suspeitas ou barreiras culturais que inviabilizem a interação. Um dicionário em Python pode ser usado para mapear expressões modernas para correspondentes mais antigas, permitindo ao programa servir como um tradutor básico que auxilia o viajante a se fazer entender.

No código a seguir, vamos criar um dicionário em que cada chave representa uma expressão moderna e cada valor é o seu equivalente em uma linguagem antiga ou histórica. O usuário poderá inserir uma frase composta por várias palavras. O programa tentará traduzir cada palavra ou expressão encontrada no dicionário. Se a palavra existir no mapeamento, substitui-a pela versão antiga; se não, mantém a palavra original. Ao final, o resultado é uma frase que reflete a linguagem da época de destino, ajudando o viajante a se comunicar com os habitantes locais sem causar estranheza.

```
# Define um dicionário de traduções, mapeando expressões modernas para
equivalentes antigos.
# Este conjunto pode ser expandido conforme a necessidade, adicionando novas
chaves e valores.
traducao = {
    "computador": "artefato mecânico de cálculo",
    "internet": "rede de mensageiros",
    "oi": "saudações",
    "tchau": "despeço-me",
    "carro": "carruagem sem cavalos",
    "obrigado": "minhas reverências",
    "problema": "contratempo"
}

# Solicita ao usuário uma frase que utiliza expressões modernas.
frase_moderna = input("Insira uma frase usando termos modernos: ")

# Divide a frase em palavras, assumindo que a separação é feita por espaços.
palavras = frase_moderna.split()

# Cria uma lista para armazenar a versão traduzida das palavras.
frase_traduzida = []

# Percorre cada palavra da lista original. Para cada uma, verifica se existe uma
chave no dicionário de tradução.
# Se existir, substitui pela expressão antiga; caso contrário, mantém a
palavra original.
for palavra in palavras:
    if palavra in traducao:
        frase_traduzida.append(traducao[palavra])
    else:
        frase_traduzida.append(palavra)

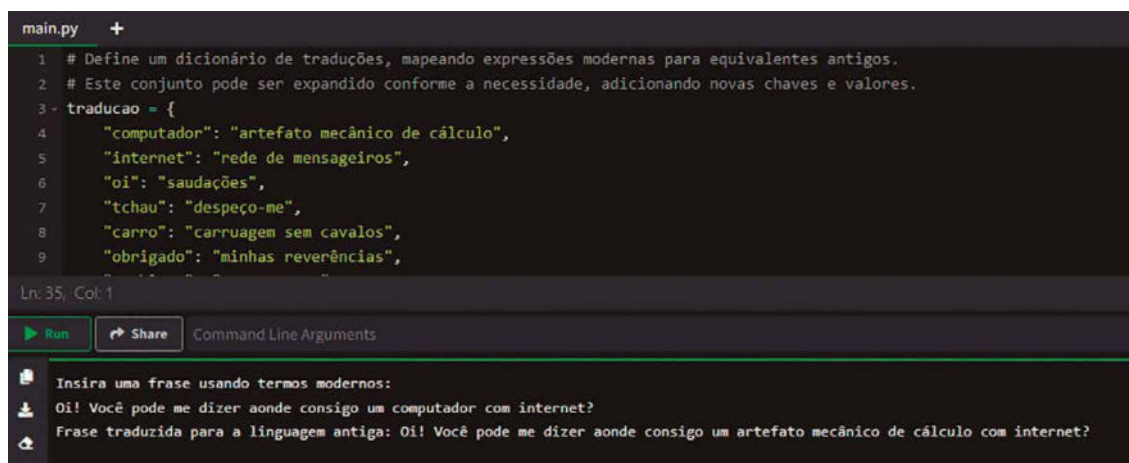
# Junta as palavras traduzidas de volta em uma frase única, usando espaço
como separador.
resultado = " ".join(frase_traduzida)

# Exibe a frase traduzida, permitindo ao viajante usá-la ao chegar na época
de destino.
print("Frase traduzida para a linguagem antiga:", resultado)
```

Figura 103

No início do código, definimos o dicionário `traducao`, em que cada chave é uma expressão moderna comum nos dias de hoje, e o valor correspondente é sua versão antiga. Para tornar essa

simulação mais autêntica, poderiam ser usadas expressões de idiomas arcaicos ou termos históricos reais, ajustando assim o dicionário a um período específico. Em um contexto de viagem no tempo, o viajante pode carregar esse mapeamento em seu dispositivo, pronto para converter qualquer frase que precise comunicar.



```

main.py +
1 # Define um dicionário de traduções, mapeando expressões modernas para equivalentes antigos.
2 # Este conjunto pode ser expandido conforme a necessidade, adicionando novas chaves e valores.
3 - traducao = {
4     "computador": "artefato mecânico de cálculo",
5     "internet": "rede de mensageiros",
6     "oi": "saudações",
7     "tchau": "despeço-me",
8     "carro": "carruagem sem cavalos",
9     "obrigado": "minhas reverências",
10    }
Ln: 35, Col: 1
Run Share Command Line Arguments
Insira uma frase usando termos modernos:
Oi! Você pode me dizer aonde consigo um computador com internet?
Frase traduzida para a linguagem antiga: Oi! Você pode me dizer aonde consigo um artefato mecânico de cálculo com internet?

```

Figura 105 – Usando o tradutor com dicionário novamente. Dessa vez, nem todas as palavras foram traduzidas

O problema identificado no código atual é que o fatiamento das palavras da frase leva em conta a pontuação, fazendo com que `Oi!` não seja reconhecida como `oi` (a chave presente no dicionário), e `internet?` não seja reconhecida como `internet`. Em outras palavras, a presença de pontuações e diferença de maiúsculas e minúsculas impedem que as substituições sejam realizadas corretamente, já que **a busca no dicionário requer correspondência exata entre a palavra encontrada e a chave existente**.

Para resolver esse problema, é necessário normalizar as palavras antes de consultá-las no dicionário. Isso pode envolver converter todas as palavras para minúsculas e remover caracteres de pontuação nos extremos. Após encontrar a correspondente tradução, podemos manter a mesma pontuação final ou apenas devolver a forma traduzida sem pontuação. Aqui vamos optar por um método simples: remover pontuação apenas do final e do início da palavra, e converter tudo para minúsculas antes de consultar o dicionário. Caso a palavra original tenha pontuações e maiúsculas, ao exibir a tradução, manteremos apenas a pontuação final, já que a alteração de maiúsculas/minúsculas ao retornar à tradução pode gerar inconsistências, mas nesse contexto a tradução para a linguagem antiga pode ser inteiramente em minúsculas, uma vez que estamos mais focados na coerência da tradução do que na formatação visual da frase.

A seguir está a versão corrigida do código que lida com esse problema. Ele removerá pontuações simples no início e no fim da palavra (como `!", "?", ":", ";"`) antes de consultar o dicionário, além de converter a palavra para minúsculas. Assim, mesmo que o usuário digite `Oi!` ou `internet?`, o código identificará `oi` e `internet` corretamente e as traduzirá se existirem no dicionário.

```
import string

# Define um dicionário de traduções, agora o uso de minúsculas é padronizado para
garantir correspondência.
traducao = {
    "computador": "artefato mecânico de cálculo",
    "internet": "rede de mensageiros",
    "oi": "saudações",
    "tchau": "despeço-me",
    "carro": "carruagem sem cavalos",
    "obrigado": "minhas reverências",
    "problema": "contratempo"
}

# Solicita ao usuário uma frase
frase_moderna = input("Insira uma frase usando termos modernos: ")

# Divide a frase em palavras
palavras = frase_moderna.split()

# Lista para armazenar a frase traduzida
frase_traduzida = []

for palavra in palavras:
    # Armazena a pontuação inicial e final caso exista
    # Assim, se a palavra for "Oi!", remove-se '!' para consulta no dicionário
    # e depois podemos recolocar a pontuação no final.

    # Identifica pontuação inicial e final:
    inicio = ""
    fim = ""

    # Enquanto o primeiro caractere da palavra for pontuação, remove do início e
guarda
    while len(palavra) > 0 and palavra[0] in string.punctuation:
        inicio += palavra[0]
        palavra = palavra[1:]

    # Enquanto o último caractere da palavra for pontuação, remove do final e
guarda
    while len(palavra) > 0 and palavra[-1] in string.punctuation:
        fim = palavra[-1] + fim
        palavra = palavra[:-1]

    # Converte a palavra (sem pontuações externas) para minúscula para consulta
    palavra_lower = palavra.lower()

    # Verifica se existe no dicionário
    if palavra_lower in traducao:
        # Usa a tradução encontrada
        palavra_traduzida = traducao[palavra_lower]
    else:
        # Caso não haja tradução, mantém a palavra original (sem alterar
        maiúsculas)
        # Observe que perdemos a distinção de maiúsculas/minúsculas, já que
        removemos a pontuação e
```

```
# analisamos a palavra em minúsculas. Para simplificar, retornaremos a
palavra original aqui.
palavra_traduzida = palavra if palavra else ""

# Reanexa pontuações iniciais e finais
palavra_final = inicio + palavra_traduzida + fim

# Adiciona à lista da frase traduzida
frase_traduzida.append(palavra_final)

# Reconstrói a frase traduzida
resultado = " ".join(frase_traduzida)

print("Frase traduzida para a linguagem antiga:", resultado)
```

Figura 106

Vamos fazer uma explicação detalhada das mudanças:

- Foi importado o módulo `string` para acessar `string.punctuation`, que contém uma lista de caracteres de pontuação. Essa estratégia facilita identificar pontuações no início e no fim das palavras.
- Antes de consultar o dicionário, a palavra é limpa de pontuações no começo e no final. Essa remoção é feita por meio de laços `while` que verificam se o primeiro ou último caractere é pontuação. Assim, `Oi!` torna-se `Oi` para consulta, e `internet?` torna-se `internet`.
- A palavra é convertida para minúsculas antes da consulta ao dicionário, assegurando que `Oi` e `oi` sejam tratados igualmente, permitindo a correspondência.
- Caso a palavra seja encontrada no dicionário, é usada sua tradução. Caso contrário, a palavra original é mantida. Por simplicidade, não tentamos restaurar o padrão de maiúsculas/minúsculas original da palavra, pois isso tornaria o código mais complexo.
- Após traduzir a palavra, reanexamos quaisquer pontuações iniciais ou finais que foram removidas, assegurando que, se a original era `Oi!`, a traduzida também mantenha a `!` no final, resultando em saudações!

Dessa forma, o problema com a falta de correspondência de chaves devido à pontuação e diferenças de maiúsculas e minúsculas é resolvido, tornando a tradução mais precisa e coerente. A figura a seguir mostra o resultado da execução dessa nova versão do código. Todas as palavras da frase foram corretamente traduzidas.

```
main.py +
1 import string
2
3 # Define um dicionário de traduções, agora o uso de minúsculas é padronizado para garantir correspondência.
4 traducaao = {
5     "computador": "artefato mecânico de cálculo",
6     "internet": "rede de mensageiros",
7     "oi": "saudações",
8     "tchau": "despeço-me",
9     "carro": "carruagem sem cavalos",
10 }
Ln 65, Col 1
Run Share Command Line Arguments
Insira uma frase usando termos modernos:
Oi! Você pode me dizer aonde consigo um computador com internet?
Frase traduzida para a linguagem antiga: saudações! Você pode me dizer aonde consigo um artefato mecânico de cálculo com rede de mensageiros?
** Process exited - Return Code: 0 **
Press Enter to exit terminal
```

Figura 107 – Usando uma nova versão do tradutor. Dessa vez, todas as palavras foram traduzidas

6 ENTRADA E SAÍDA DE DADOS

Em computação, os conceitos de entrada, saída e dados estão diretamente relacionados à interação entre um programa e seu ambiente, seja ele o usuário, outros sistemas ou os dispositivos de hardware.

Entrada refere-se ao processo de fornecer informações para o programa, permitindo que ele receba os dados necessários para realizar suas tarefas. Esses dados podem ser fornecidos de diversas maneiras, como por meio do teclado, arquivos, sensores ou conexões de rede.

Saída, por sua vez, é o processo de devolver informações, permitindo que o programa comunique os resultados de suas operações ao ambiente externo. Esses resultados podem ser exibidos em um monitor, gravados em um arquivo ou enviados para outro sistema.

Dados são as informações que transitam nesses processos, podendo ser números, textos, imagens ou qualquer tipo de conteúdo estruturado ou não estruturado. A figura a seguir mostra esse relacionamento entre a entrada e saída de dados de um programa:

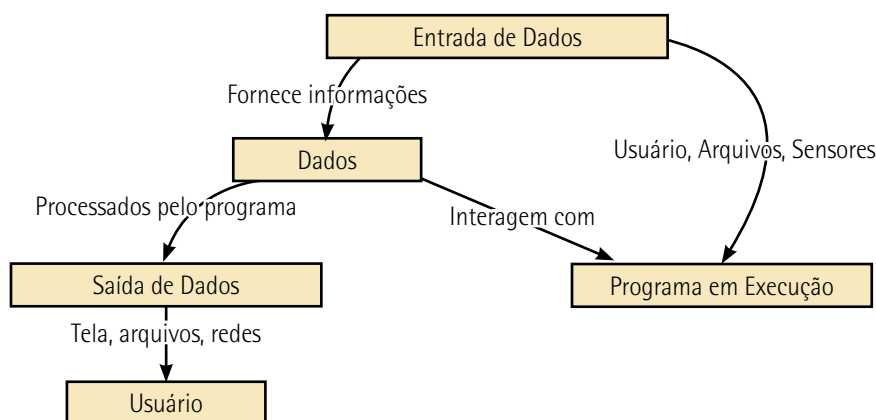


Figura 108 – Relacionamento entre entrada e saída de dados em um programa

Em Python, entrada e saída de dados são tratadas de forma simples e intuitiva, o que reflete a filosofia da linguagem de tornar a programação acessível. A entrada de dados é comumente realizada por meio da função `input()`, que exibe uma mensagem ao usuário e espera que ele insira um valor pelo teclado. Esse valor é recebido como uma string e pode ser convertido para outros tipos de dados, como inteiros ou floats, dependendo das necessidades do programa. Essa funcionalidade é essencial em programas interativos, nos quais o comportamento do código depende das informações fornecidas em tempo de execução.

A saída de dados em Python é frequentemente realizada com a função `print()`, que exibe informações no console de forma clara e flexível. Essa função permite apresentar textos estáticos, valores de variáveis e resultados de cálculos, muitas vezes combinando todos esses elementos em mensagens formatadas. Python oferece várias opções para personalizar a saída, como a interpolação de strings e o uso de especificadores de formato, permitindo que o desenvolvedor controle com precisão como os dados são apresentados ao usuário.



Observação

Para que você tenha uma dimensão da relevância das entradas e saídas de dados, se você acompanhou este livro-texto desde o início, saiba que já usamos a função `input()` 51 vezes e a função `print()` 54 vezes.

Além das entradas e saídas básicas pelo teclado e console, Python oferece suporte a operações mais avançadas, como leitura e escrita em arquivos, comunicação com bancos de dados e troca de informações pela rede. Essas funcionalidades ampliam significativamente as possibilidades de interação entre o programa e seu ambiente, permitindo que ele processe grandes volumes de dados, armazene resultados para uso futuro ou se integre a sistemas maiores.

A combinação eficiente de entrada e saída é fundamental para tornar um programa útil e funcional. Ao fornecer dados para processamento e exibir resultados de forma clara e acessível, essas operações garantem que os programas sejam capazes de atender às necessidades dos usuários e resolver problemas do mundo real. Em Python, as ferramentas para entrada e saída são projetadas para facilitar essa interação, tornando-as acessíveis tanto para iniciantes quanto para desenvolvedores experientes.

6.1 Leitura de dados do teclado

A função mais comumente utilizada para capturar dados fornecidos pelo usuário é a `input()`. Essa função exibe uma mensagem no console, conhecido como `prompt`, e aguarda que o usuário insira um valor.



Observação

O console, em computação, é uma interface textual que permite a interação entre o usuário e o sistema operacional ou um programa. Ele é uma ferramenta essencial em ambientes de desenvolvimento e administração de sistemas, permitindo que comandos sejam digitados, dados sejam inseridos e resultados sejam exibidos diretamente, sem a necessidade de interfaces gráficas.

Funcionalmente, o console atua como uma janela de texto onde o usuário pode digitar comandos e visualizar saídas. Ele captura as entradas fornecidas pelo teclado e exibe as saídas, geralmente sob a forma de texto, enviadas pelo programa. Essa simplicidade o torna uma ferramenta poderosa, sobretudo em sistemas operacionais baseados em texto, como terminais Linux ou o Prompt de Comando no Windows.

O console é usualmente chamado de prompt porque, em sua essência, ele apresenta ao usuário uma indicação visual ou textual de que está pronto para receber comandos ou entradas. Essa indicação é conhecida como o "prompt do sistema" e é uma parte fundamental de interfaces de linha de comando, como o terminal em sistemas Unix/Linux, o Prompt de Comando no Windows ou mesmo o console interativo do Python. É a famosa "tela preta" ou "tela azul" que os profissionais de tecnologia da informação interagem pelo teclado (texto).

O termo **prompt** vem do inglês e significa pronto ou solicitação. No contexto do console, refere-se à mensagem exibida que solicita ao usuário que insira algo. Tradicionalmente, o prompt é uma combinação de símbolos, textos ou informações, como o nome do usuário, o diretório atual ou um simples caractere (\$, > ou >>>), dependendo do sistema ou programa. Essa mensagem não é apenas estética; ela informa ao usuário que o console está ativo, aguardando comandos ou dados.

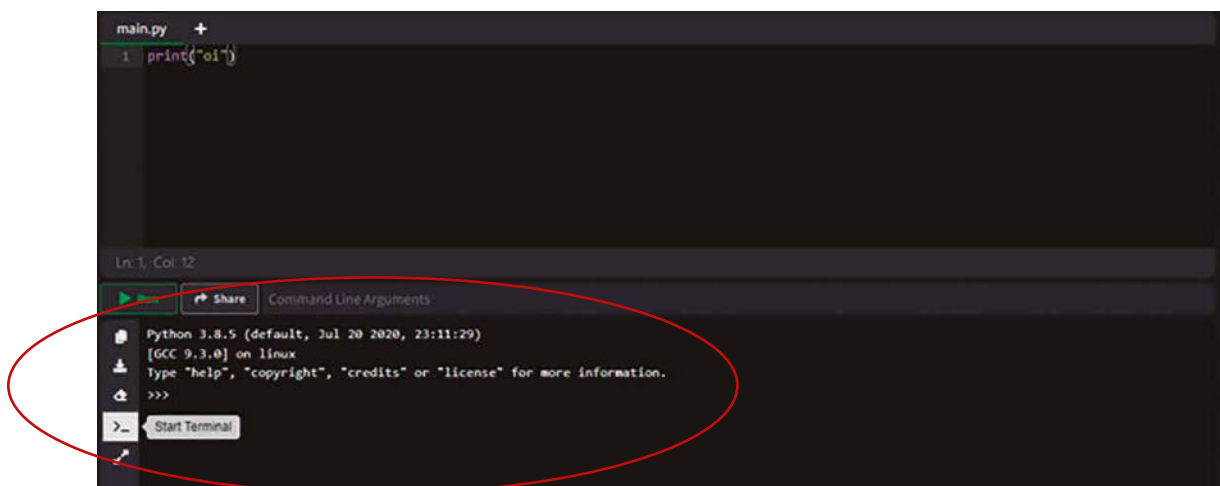


Figura 109 – Nosso console ou prompt em destaque (círculo vermelho)

Quando o usuário digita a informação e pressiona a tecla Enter, a `input()` retorna o valor fornecido como uma string. Isso significa que, independentemente do que for inserido, um número, texto ou caracteres especiais, o dado é sempre interpretado inicialmente como texto. Por exemplo, se o usuário digitar 42, o valor retornado será a string 42, e não o número 42.

Em muitos casos, os dados fornecidos pelo usuário precisam ser convertidos para outro tipo antes de serem utilizados no programa. Python oferece funções como `int()` para converter strings em números inteiros, `float()` para números decimais e `bool()` para valores booleanos. Já usamos as essas funções nos códigos-fonte anteriormente e, como vimos, as conversões são vitais quando o programa precisa realizar cálculos matemáticos ou tomar decisões baseadas em valores numéricos.

Preparar uma viagem no tempo levanta uma reflexão sobre por que se quer chegar naquela época. Talvez o motivo seja estudar um evento histórico em primeira mão, recuperar um objeto precioso, encontrar um personagem lendário ou simplesmente testemunhar um acontecimento marcante. Ao tornar o processo interativo, solicitando informações diretamente do usuário, é possível adaptar a experiência e gerar uma configuração personalizada, tornando a viagem temporal mais coerente e significativa. Ao final, o código terá coletado o ano de destino e o objetivo principal, permitindo que tais dados sejam posteriormente utilizados em cálculos, exibições ou outras funcionalidades relacionadas à jornada temporal.

A seguir está um exemplo de código que utiliza a função `input()` para coletar dados do usuário. O programa pedirá o ano de destino, o objetivo da viagem e se o usuário deseja retornar ao presente imediatamente ou permanecer um tempo naquela época, criando assim um cenário imaginário e adaptável.

```
from datetime import datetime

# Obtém o ano atual do sistema, representando o "presente" do viajante.
ano_atual = datetime.now().year

# Solicita o ano de destino. O usuário insere um ano, e o programa converte para inteiro.
# Caso o usuário insira um valor menor que o ano atual, significa uma viagem ao passado;
# se for maior, ao futuro; se igual, permanece no presente.
ano_destino = int(input("Insira o ano de destino da sua viagem no tempo: "))

# Solicita o objetivo da viagem. Aqui, não há uma estrutura rígida, o usuário pode digitar qualquer texto explicando suas intenções.
objetivo = input("Qual é o objetivo principal desta viagem? (ex: testemunhar um evento, encontrar alguém, estudar a cultura) ")

# Pergunta se o usuário deseja retornar imediatamente após atingir o ano de destino, ou se vai permanecer um tempo lá.
# Aqui, uma resposta simples como 's' ou 'n' pode determinar o comportamento.
permanecer = input("Você deseja permanecer um tempo no destino antes de retornar ao presente? (s/n) ")
```

```
# Exibe um resumo das informações coletadas. Isso permite ao usuário verificar se
os dados inseridos fazem sentido.
print("\n--- Configuração da Viagem Temporal ---")
print(f"Ano atual: {ano_atual}")
print(f"Ano de destino: {ano_destino}")
print(f"Objetivo da viagem: {objetivo}")

# Para informar a direção temporal da viagem, verifica-se a relação entre ano_
destino e ano_atual.
if ano_destino > ano_atual:
    print("Você está planejando uma viagem ao futuro.")
elif ano_destino < ano_atual:
    print("Você está preparando uma jornada ao passado.")
else:
    print("Você planeja permanecer no ano atual, talvez observando eventos
atuais com outro olhar.")

# Sobre a permanência, informa o usuário com base na resposta 's' ou 'n'.
if permanecer.lower() == 's':
    print("Você decidiu permanecer um tempo no destino antes de retornar. Ajuste
seus recursos e aproveite a estadia.")
else:
    print("Você optou por retornar imediatamente após a chegada, minimizando sua
exposição ao período histórico.")

# Ao final, as informações coletadas podem ser utilizadas em cálculos
posteriores,
# por exemplo, estimando a energia necessária, ajustando a interface do
dispositivo temporal,
# ou planejando roteiros de visitas com base no objetivo descrito.
```

Figura 110

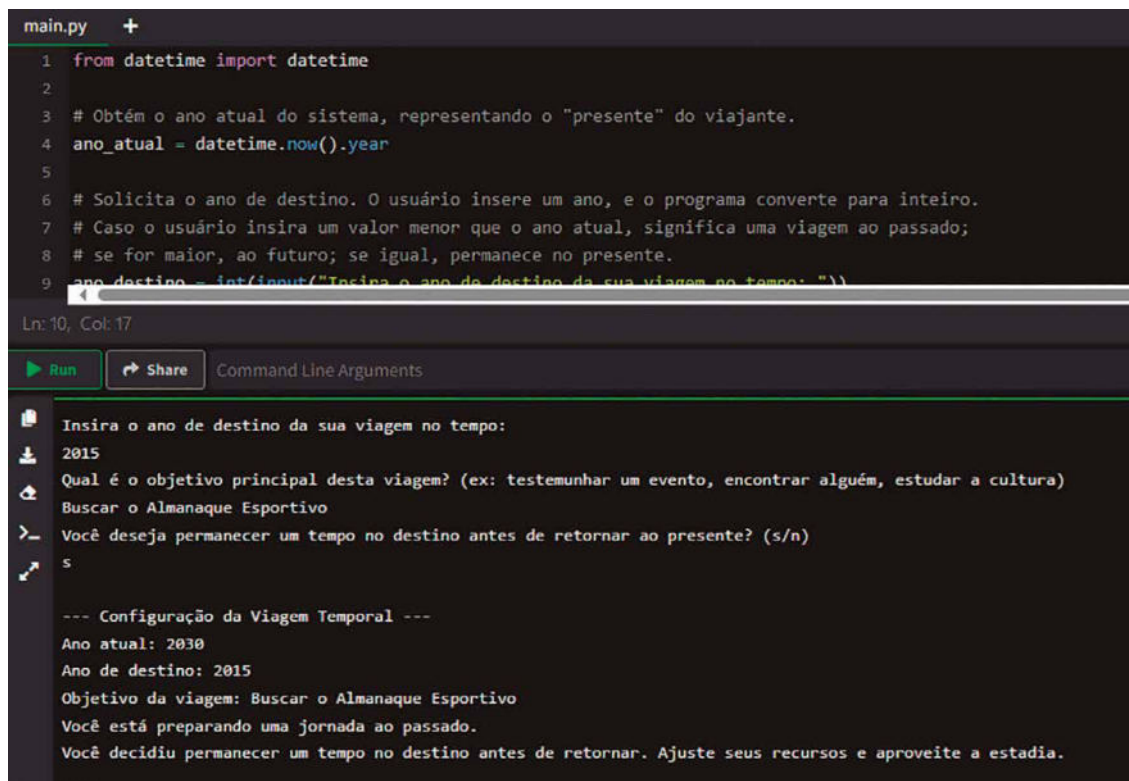
Na primeira parte do código, utilizamos novamente o módulo `datetime` para obter o ano atual, definindo uma referência temporal. Em seguida, solicitamos ao usuário que informe o ano de destino. Essa é uma etapa essencial, pois determina o tipo de salto temporal: um deslocamento ao passado, ao futuro ou a permanência no presente. A resposta do usuário é convertida para inteiro, permitindo ao programa analisar numericamente a diferença entre o ano atual e o ano de destino.

Em seguida, solicita-se ao usuário que descreva o objetivo principal da viagem. Essa informação é livre, não há conversão de tipos, trata-se de um texto que pode guiar o viajante ou o software da máquina do tempo na escolha das ferramentas necessárias.

O programa então pergunta se o usuário pretende permanecer um tempo no destino antes de voltar ao presente. Essa pergunta simula a possibilidade de o viajante interagir com o ambiente histórico, coletar informações, realizar estudos ou cumprir missões mais complexas.

Ao final, o programa imprime um resumo, permitindo ao viajante conferir se todos os dados inseridos estão corretos. Dependendo da relação entre `ano_atual` e `ano_destino`, o programa informa se a viagem será ao passado, ao futuro ou se ficará no presente. Além disso, analisa a resposta

sobre a permanência, exibindo uma mensagem condizente com a escolha. A figura a seguir apresenta a execução desse programa na tela do console:



```
main.py +
1 from datetime import datetime
2
3 # Obtém o ano atual do sistema, representando o "presente" do viajante.
4 ano_atual = datetime.now().year
5
6 # Solicita o ano de destino. O usuário insere um ano, e o programa converte para inteiro.
7 # Caso o usuário insira um valor menor que o ano atual, significa uma viagem ao passado;
8 # se for maior, ao futuro; se igual, permanece no presente.
9 ano_destino = int(input("Insira o ano de destino da sua viagem no tempo: "))

Ln: 10, Col: 17

Run Share Command Line Arguments

Insira o ano de destino da sua viagem no tempo:
2015
Qual é o objetivo principal desta viagem? (ex: testemunhar um evento, encontrar alguém, estudar a cultura)
Buscar o Almanaque Esportivo
> Você deseja permanecer um tempo no destino antes de retornar ao presente? (s/n)
s

--- Configuração da Viagem Temporal ---
Ano atual: 2030
Ano de destino: 2015
Objetivo da viagem: Buscar o Almanaque Esportivo
Você está preparando uma jornada ao passado.
Você decidiu permanecer um tempo no destino antes de retornar. Ajuste seus recursos e aproveite a estadia.
```

Figura 111 – Resultado da execução do programa que coleta diversas entradas

Ao longo dessa interação, o uso de `input()` garante que o programa seja dinâmico e responda às escolhas do usuário, tornando a experiência mais orgânica. Contudo, é importante que o desenvolvedor lide com potenciais erros durante a conversão, pois a entrada de dados pode conter valores inválidos. Por exemplo, tentar converter a string `abc` em um número resultará em um erro. Para evitar que o programa falhe, é recomendável o uso de blocos `try-except` para tratar exceções de forma controlada.

A preparação de uma viagem no tempo pode se tornar ainda mais realista e robusta ao implementar diversas extensões que tornam o código mais confiável e repleto de recursos. Uma das primeiras melhorias é a **validação de dados**. Não é desejável que o usuário insira um ano de destino inviável, como uma letra em vez de um número, ou um valor absurdo. Também é possível incluir cálculos adicionais, como estimar a energia necessária para a viagem com base na distância temporal entre o ano atual e o ano de destino. Além disso, se o programa já contiver outras partes do sistema, como um dicionário de expressões antigas ou uma agenda temporal, o código pode consultar essas estruturas para informar o usuário sobre condições linguísticas no destino ou compromissos previamente marcados.

A seguir está um exemplo ampliado do código anterior. Agora, ele utiliza estruturas de `try-except` para tratar exceções, garantindo que o usuário insira anos de destino coerentes. Também simula o cálculo de energia necessária para o salto, presumindo que quanto maior a diferença temporal, maior o custo

energético. Por fim, o código mostra como seria possível acessar outros componentes do programa, como um dicionário de expressões antigas para sugerir adaptações linguísticas ou uma hipotética agenda temporal que já exista. Esses elementos extras podem ser ajustados conforme a necessidade do sistema maior, mantendo a ideia central de uma preparação interativa e confiável para a jornada no tempo.

```
from datetime import datetime

# Ano atual obtido do sistema, utilizado como referência para cálculos.
ano_atual = datetime.now().year

# Simula a existência de outras partes do programa às quais se pode conectar.
# Por exemplo, um dicionário de expressões antigas (citado em exercícios anteriores).
expressões_antigas = {
    "computador": "artefato mecânico de cálculo",
    "internet": "rede de mensageiros"
}

# Simula a existência de uma agenda temporal previamente estabelecida, com anos e compromissos.
agenda_temporal = {
    1920: ["Conferência com pesquisadores históricos", "Acompanhar descoberta arqueológica"],
    2050: ["Observar cidade futurista", "Experimentar culinária sintética"]
}

# Função para solicitar o ano de destino com validação e tratamento de exceções.
def obter_ano_destino():
    while True:
        entrada = input("Insira o ano de destino da sua viagem no tempo: ")
        try:
            ano = int(entrada)
            if ano <= 0:
                print("Por favor, insira um ano positivo válido.")
            else:
                return ano
        except ValueError:
            print("Entrada inválida. Por favor, insira um número inteiro representando o ano.")

# Função para solicitar uma resposta sim/não, tratando erros de digitação.
def obter_resposta_sim_nao(mensagem):
    while True:
        resposta = input(mensagem + " (s/n): ").strip().lower()
        if resposta in ['s', 'n']:
            return resposta
        else:
            print("Responda apenas com 's' para sim ou 'n' para não.")

# Início do programa principal:
ano_destino = obter_ano_destino()

# Solicita o objetivo da viagem, sem validação complexa, pois é um texto livre.
objetivo = input("Qual é o objetivo principal desta viagem? (ex: testemunhar um evento, encontrar alguém, estudar a cultura) ")
```

```
# Pergunta se o usuário deseja permanecer no destino antes de retornar.
permanecer = obter_resposta_sim_nao("Você deseja permanecer um tempo no destino
antes de retornar ao presente")

# Exibição de dados iniciais coletados.
print("\n--- Configuração da Viagem Temporal ---")
print(f"Ano atual: {ano_atual}")
print(f"Ano de destino: {ano_destino}")
print(f"Objetivo da viagem: {objetivo}")

if ano_destino > ano_atual:
    print("Você está planejando uma viagem ao futuro.")
elif ano_destino < ano_atual:
    print("Você está preparando uma jornada ao passado.")
else:
    print("Você planeja permanecer no ano atual, talvez observando eventos
atuais sob outra perspectiva.")

if permanecer == 's':
    print("Você decidiu permanecer um tempo no destino antes de retornar. Ajuste
seus recursos e aproveite a estadia.")
else:
    print("Você optou por retornar imediatamente após a chegada, minimizando sua
exposição ao período histórico.")

# Cálculo adicional: Estima a energia necessária para o salto, supondo que cada
ano de diferença custa 10 unidades.
diferenca = abs(ano_destino - ano_atual)
energia_necessaria = diferenca * 10 # Fator arbitrário.
print(f"\nEnergia necessária para o salto temporal: {energia_necessaria}
unidades.")

# Conexão com outras partes do programa: se o destino tiver expressões antigas
disponíveis, pode-se alertar o usuário.
if expressões_antigas:
    print("\nLembre-se de adaptar sua linguagem ao chegar lá. Algumas expressões
modernas têm equivalentes antigos:")
    for moderna, antiga in expressões_antigas.items():
        print(f"'{moderna}' pode ser dito como '{antiga}' no destino.")

# Consulta à agenda temporal: se o destino estiver na agenda, mostra
compromissos.
if ano_destino in agenda_temporal:
    print(f"\nVocê tem compromissos agendados no ano {ano_destino}:")
    for idx, comp in enumerate(agenda_temporal[ano_destino], start=1):
        print(f"{idx}. {comp}")
else:
    print(f"\nNenhum compromisso pré-cadastrado para o ano {ano_destino}.")

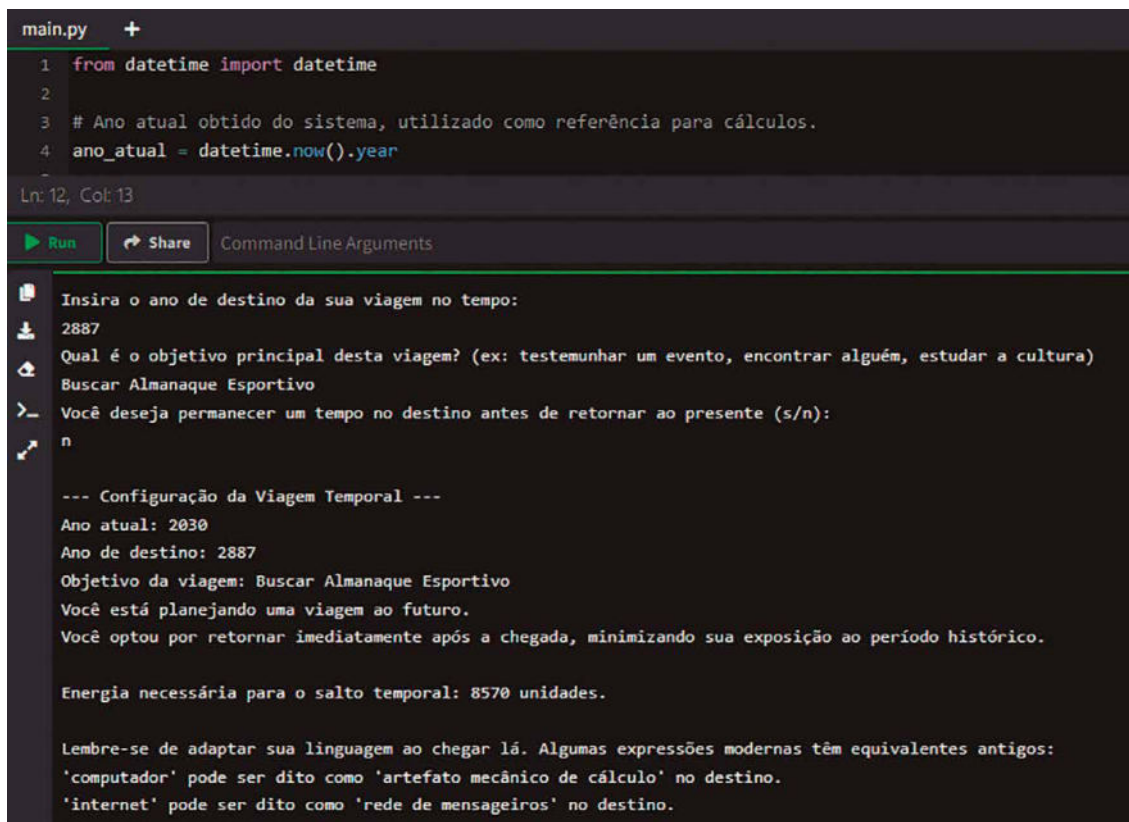
# Ao final, o código reuniu dados do usuário, validou, executou cálculos e se
conectou a outras partes
# do sistema, criando assim uma experiência mais rica e segura para o viajante
do tempo.
```

Figura 112

Nessa versão ampliada do código, há várias melhorias:

- Uso de `try/except` no recebimento do ano de destino, garantindo que apenas valores inteiros positivos sejam aceitos. Caso o usuário insira algo inválido, a solicitação é feita novamente.
- Uma função auxiliar para obter respostas do tipo sim/não, também garantindo coerência na entrada.
- Inclusão de um cálculo adicional para estimar a energia necessária da viagem, relacionando a distância temporal à quantidade de energia.
- Conexão com outras partes do sistema, aqui simuladas por dicionários `expressões_antigas` e `agenda_temporal`. O primeiro sugere adaptações linguísticas com base em um mapeamento pré-definido, e o segundo verifica se há compromissos agendados no ano de destino.

A figura 113 apresenta a execução do novo programa. Note que inserimos um valor de data (2887) em um formato numérico já esperado e a execução seguiu até o final. Já na execução da figura 114 inserimos os valores `abc`, `efg` e `hij`, que não podem ser convertidos para anos. Graças ao tratamento de erros que implementamos com `try/except`, esses valores de entrada foram rejeitados pelo programa que solicitou nova digitação ao usuário.



```
main.py +
1 from datetime import datetime
2
3 # Ano atual obtido do sistema, utilizado como referência para cálculos.
4 ano_atual = datetime.now().year
-
Ln: 12, Col: 13
Run Share Command Line Arguments

Insira o ano de destino da sua viagem no tempo:
2887
Qual é o objetivo principal desta viagem? (ex: testemunhar um evento, encontrar alguém, estudar a cultura)
Buscar Almanaque Esportivo
>_ Você deseja permanecer um tempo no destino antes de retornar ao presente (s/n):
n

--- Configuração da Viagem Temporal ---
Ano atual: 2030
Ano de destino: 2887
Objetivo da viagem: Buscar Almanaque Esportivo
Você está planejando uma viagem ao futuro.
Você optou por retornar imediatamente após a chegada, minimizando sua exposição ao período histórico.

Energia necessária para o salto temporal: 8570 unidades.

Lembre-se de adaptar sua linguagem ao chegar lá. Algumas expressões modernas têm equivalentes antigos:
'computador' pode ser dito como 'artefato mecânico de cálculo' no destino.
'internet' pode ser dito como 'rede de mensageiros' no destino.
```

Figura 113 – Execução com dados válidos

```

main.py +
1 from datetime import datetime
2
3 # Ano atual obtido do sistema, utilizado como referência para cálculos.
4 ano_atual = datetime.now().year
-
Ln: 12, Col: 13
[Stop] [Share] Command Line Arguments

Insira o ano de destino da sua viagem no tempo:
abc
Entrada inválida. Por favor, insira um número inteiro representando o ano.
Insira o ano de destino da sua viagem no tempo:
efg
Entrada inválida. Por favor, insira um número inteiro representando o ano.
Insira o ano de destino da sua viagem no tempo:
hij
Entrada inválida. Por favor, insira um número inteiro representando o ano.
Insira o ano de destino da sua viagem no tempo:

```

Figura 114 – Execução com dados inválidos para o ano

A máquina do tempo, sendo uma tecnologia tão delicada e poderosa, não deve ser acionada sem um mínimo de segurança. Imagine o risco de qualquer pessoa ter acesso irrestrito a viagens temporais, podendo interferir na história, criar paradoxos ou obter informações do futuro para seu próprio benefício. É por isso que, antes de iniciar a máquina, uma autenticação simples se faz necessária. Embora não seja uma segurança intransponível, um código de acesso conhecido apenas pelo operador autorizado já dificulta que indivíduos não qualificados utilizem o dispositivo para causar desequilíbrios irreparáveis no continuum espaço-tempo.

A seguir está um código em Python que ilustra essa ideia. O programa solicita que o usuário insira um código de acesso. Se o código estiver correto, o programa permite que a máquina do tempo seja acionada; caso contrário, recusa o acesso. Para tornar o exemplo um pouco mais robusto, o programa oferece um número limitado de tentativas, evitando tentativas infinitas e consecutivas de adivinhação do código. Além disso, cada linha do código é acompanhada de explicações e justificativas, fundamentando o uso de cada função, condição e variável, relacionando-as com o contexto da viagem temporal.

```

codigo_correto = "Delta42" # Este é o código de acesso conhecido pelo operador
                           # autorizado da máquina do tempo.
                           # Aqui, escolhemos uma string "Delta42" como exemplo,
                           # mas poderia ser qualquer sequência
                           # complexa, incluindo caracteres especiais, números e
                           # letras maiúsculas.

tentativas = 3 # O número total de tentativas permitidas para inserir o código.
               # Essa limitação cria uma barreira simples contra a força bruta,
               # desestimulando tentativas aleatórias
               # e sucessivas de descobrir o código por quem não o conhece.

while tentativas > 0: # Enquanto ainda houver tentativas disponíveis, o programa
                    # continua solicitando o código.

```

```
entrada = input("Insira o código de acesso da máquina do tempo: ")
# A função input() aguarda o usuário digitar algo no teclado e pressionar
Enter.
# O valor digitado é armazenado na variável 'entrada'. Aqui, representa a
tentativa atual de autenticação.

if entrada == codigo_correto:
    # Esta condição verifica se a entrada do usuário corresponde exatamente
    ao código correto.
    # A comparação == checa a igualdade entre as duas strings.
    print("Código correto. Acesso concedido.")
    # Caso o código digitado seja o correto, o programa exibe uma mensagem
    de sucesso.
    # Neste ponto, poderíamos chamar outras funções do sistema de viagem
    no tempo, iniciar preparativos,
    # exibir menus com opções de configuração ou acionamento da máquina.
    break
    # O comando break encerra o loop, uma vez que a autenticação foi
    bem-sucedida, não há motivo para pedir novamente.
else:
    # Caso o código esteja incorreto, o programa entra neste else.
    tentativas -= 1
    # Reduzimos o número de tentativas disponíveis em uma unidade, punindo o
    erro e aproximando o usuário do bloqueio.
    if tentativas > 0:
        # Se ainda há tentativas disponíveis após o erro, o programa informa
        quantas restam.
        # Isso dá feedback ao usuário, avisando que se continuar errando,
        perderá o acesso.
        print(f"Código incorreto. Você ainda tem {tentativas} tentativa(s).")
    else:
        # Caso as tentativas tenham chegado a zero, o programa informa que o
        acesso será negado.
        print("Todas as tentativas esgotadas. Acesso negado.")
        # Nesta situação, poderíamos implementar um bloqueio temporário da
        máquina, emitir alertas,
        # ou acionar protocolos de segurança mais rigorosos, dependendo da
        complexidade desejada.
```

Figura 115

A lógica começa definindo o código correto dentro da variável `codigo_correto`, que no caso é Delta42. Esse código simboliza a chave de acesso, algo que somente o operador autorizado deveria conhecer. Logo a seguir, estabelece-se uma variável `tentativas` com valor 3, limitando as oportunidades de acerto.

A estrutura `while tentativas > 0` cria um loop que só continua rodando se o usuário ainda tiver tentativas disponíveis. Dentro do loop, a função `input()` pergunta sobre o código ao usuário. O valor digitado é armazenado em `entrada`. Em seguida, um `if` compara `entrada` com `codigo_correto`. Se forem iguais, a mensagem de sucesso é exibida, o acesso é concedido e o loop é interrompido com `break`. Caso contrário, o bloco `else` executa, decrementando a variável `tentativas` em um. Se ainda houver tentativas após o erro, o programa informa quantas restam, dando ao operador uma segunda chance. Se não restarem tentativas, a mensagem final de acesso negado é

impressa, sinalizando que o programa não prosseguirá com a ativação da máquina. A figura 116 mostra uma execução com três tentativas inválidas e, portanto, o acesso à máquina do tempo foi negado. Já a figura 117 ilustra um acesso com a senha correta e – ato contínuo – validado pelo sistema.

```
main.py +
1  codigo_correto = "Delta42" # Este é o código de acesso conhecido pelo operador autorizado da máquina do tempo.
2                                # Aqui, escolhemos uma string "Delta42" como exemplo, mas poderia ser qualquer sequência
3                                # complexa, incluindo caracteres especiais, números e letras maiúsculas.
4
5  tentativas = 3 # O número total de tentativas permitidas para inserir o código.
6                # Essa limitação cria uma barreira simples contra a força bruta, desestimulando tentativas aleatórias
7                # e sucessivas de descobrir o código por quem não o conhece.
8
Ln: 36, Col: 1
▶ Run ▶ Share Command Line Arguments
❏ Insira o código de acesso da máquina do tempo:
📄 Abracadabra
📄 Código incorreto. Você ainda tem 2 tentativa(s).
📄 Insira o código de acesso da máquina do tempo:
📄 Shazamm
📄 Código incorreto. Você ainda tem 1 tentativa(s).
📄 Insira o código de acesso da máquina do tempo:
📄 Abre-te Sésamo
📄 Todas as tentativas esgotadas. Acesso negado.
```

Figura 116 – Programa que verifica o código de acesso para controlar a máquina do tempo

```
main.py +
1  codigo_correto = "Delta42" # Este é o código de acesso conhecido pelo operador au
2                                # Aqui, escolhemos uma string "Delta42" como exemplo, ma
3                                # complexa, incluindo caracteres especiais, números e le
4
5  tentativas = 3 # O número total de tentativas permitidas para inserir o código.
6                # Essa limitação cria uma barreira simples contra a força bruta, de
7                # e sucessivas de descobrir o código por quem não o conhece.
8
Ln: 36, Col: 1
▶ Run ▶ Share Command Line Arguments
❏ Insira o código de acesso da máquina do tempo:
📄 admin
📄 Código incorreto. Você ainda tem 2 tentativa(s).
📄 Insira o código de acesso da máquina do tempo:
📄 administrador
📄 Código incorreto. Você ainda tem 1 tentativa(s).
📄 Insira o código de acesso da máquina do tempo:
📄 Delta42
📄 Código correto. Acesso concedido.
```

Figura 117 – Tentativa de acesso bem-sucedida à máquina do tempo

A segurança do dispositivo não precisa ser apenas um limite fixo de tentativas, também pode implementar mecanismos inteligentes de dissuasão. Em vez de simplesmente dar três chances e, ao falhar, bloquear o acesso, o sistema pode aumentar o tempo de espera a cada erro, obrigando a pessoa sem acesso legítimo aguardar intervalos cada vez maiores antes de tentar novamente. Essa abordagem, inspirada em algoritmos de backoff exponencial, torna o processo de adivinhação do código trabalhoso e demorado, desencorajando quem não conhece a chave correta.

A seguir está o código adaptado para essa nova lógica. Dessa vez, não há um número limitado de tentativas, porém, a cada erro, o usuário recebe a informação de que deverá esperar um certo número de minutos antes de tentar novamente. Esse tempo começa pequeno, mas dobra a cada falha, tornando cada nova tentativa potencialmente muito mais demorada. Caso o usuário acerte o código, o programa dá acesso imediato, sem mais atrasos.

```
import time

codigo_correto = "Delta42"
tempo_espera = 1

while True:
    entrada = input("Insira o código de acesso da máquina do tempo: ")
    if entrada == codigo_correto:
        print("Código correto. Acesso concedido.")
        break
    else:
        # Código incorreto: informa o tempo de espera em minutos.
        print(f"Senha incorreta. Tente novamente em {tempo_espera} minuto(s).")

        # Ao invés de simplesmente fazer time.sleep(), vamos fornecer feedback
visual.
        # Convertemos minutos em segundos para a contagem regressiva.
        segundos_espera = tempo_espera * 60

        # Exibir contagem regressiva segundo a segundo.
        # Por exemplo, se tempo_espera for 1, serão 60 segundos.
        for i in range(segundos_espera, 0, -1):
            print(f"Aguardando... {i} segundo(s) restantes", end='\r')
            time.sleep(1)

        # Ao final da espera, imprimimos uma quebra de linha para limpar a última
mensagem.
        print("\nTempo de espera encerrado. Você pode tentar novamente agora.")

        # Dobra o tempo de espera para a próxima tentativa incorreta.
        tempo_espera *= 2
```

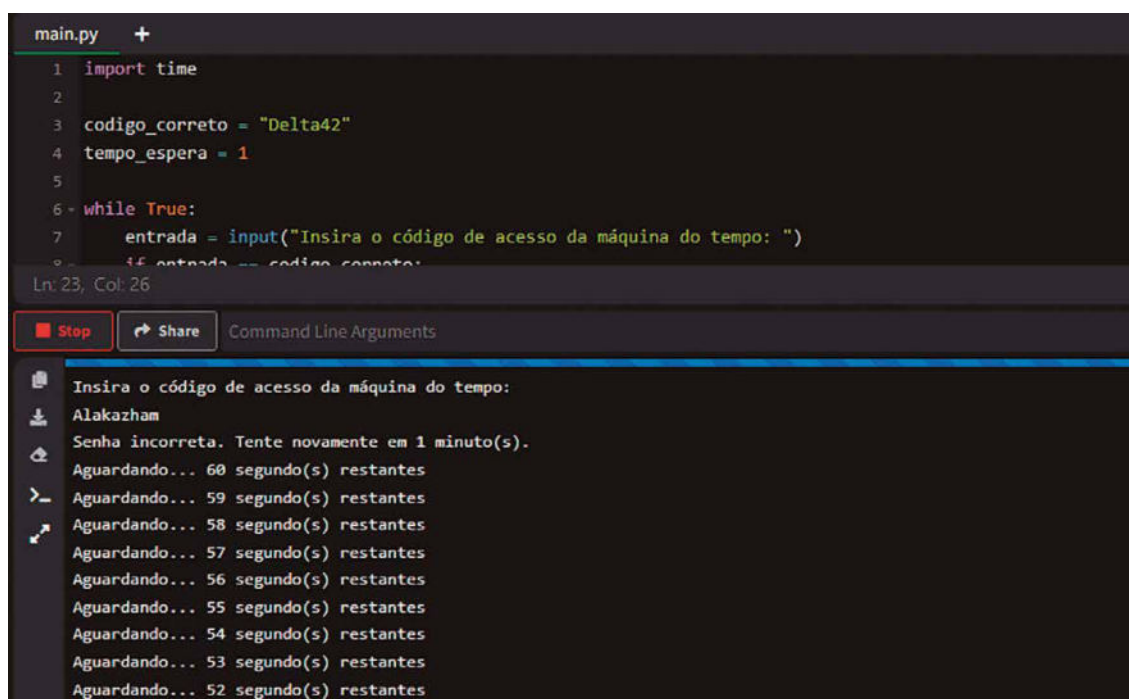
Figura 118

Em comparação ao código anterior, que limitava a três tentativas, aqui não existe mais um número máximo de chances. A restrição vem do tempo, não do número de erros. A cada vez que o usuário erra, recebe o aviso de que deverá esperar um certo tempo antes de tentar novamente. Inicialmente, esse

tempo é de apenas 1 minuto, algo facilmente tolerável. Porém, se o usuário continuar errando, o tempo de espera dobra para 2 minutos, depois para 4, 8, 16 e assim por diante. Com o crescimento exponencial desse intervalo, tornar-se-á inviável persistir em tentativas aleatórias, a menos que se esteja disposto a gastar um tempo absurdamente longo entre uma tentativa e outra.

Em vez de usar diretamente a função `time.sleep()` para pausar o programa por um longo período, o código utiliza um loop `for` para exibir uma contagem regressiva segundo a segundo. A variável `segundos_espera` converte o tempo de espera de minutos para segundos, permitindo que o programa conte de forma precisa e dinâmica. Durante cada segundo da contagem regressiva, o programa atualiza a mensagem exibida ao usuário com o número de segundos restantes, utilizando o parâmetro `end='\r'` para sobrepor a mensagem anterior no console, criando um efeito visual contínuo, caso esteja executando no ambiente do sistema operacional.

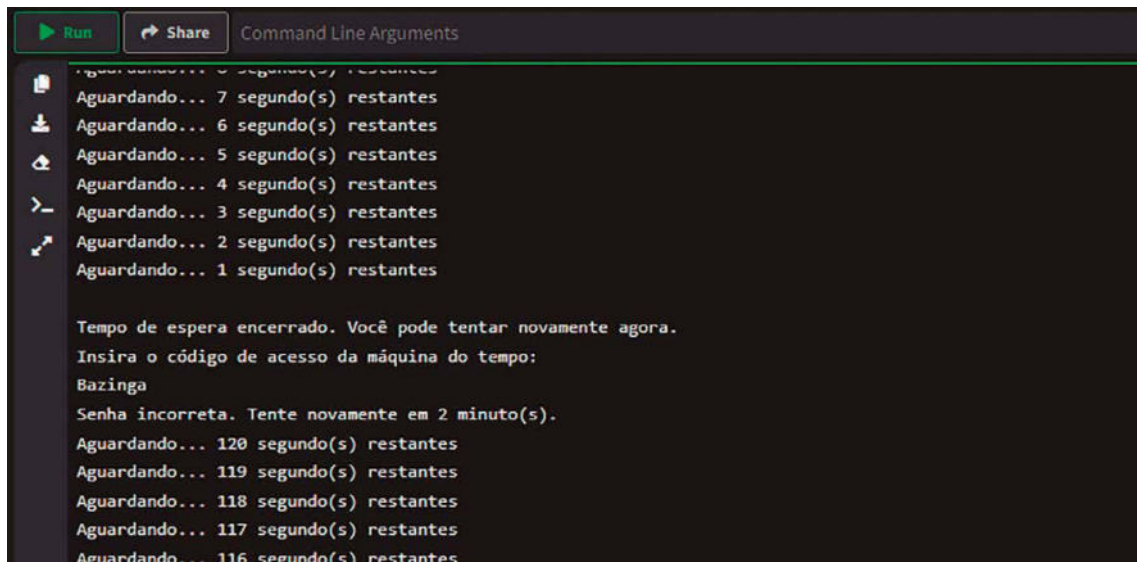
Quando a contagem regressiva atinge zero, o programa imprime uma nova linha informando que o tempo de espera terminou e que o usuário pode tentar novamente. Antes de reiniciar o processo, o tempo de espera é dobrado, aumentando a penalidade para tentativas incorretas subsequentes. Essa abordagem progressiva imita sistemas reais de segurança, nos quais o tempo entre tentativas aumenta para dificultar ataques de força bruta. A figura 119 mostra uma falha no acesso e o sistema inicia a contagem regressiva; na figura 120, uma nova tentativa com falha e o tempo de contagem aumenta; na figura 121, a tentativa foi bem-sucedida.



```
main.py +
1 import time
2
3 codigo_correto = "Delta42"
4 tempo_espera = 1
5
6 while True:
7     entrada = input("Insira o código de acesso da máquina do tempo: ")
8     if entrada == codigo_correto:
9         break
10
Ln: 23, Col: 26
[Stop] [Share] Command Line Arguments

Insira o código de acesso da máquina do tempo:
Alakazham
Senha incorreta. Tente novamente em 1 minuto(s).
Aguardando... 60 segundo(s) restantes
Aguardando... 59 segundo(s) restantes
Aguardando... 58 segundo(s) restantes
Aguardando... 57 segundo(s) restantes
Aguardando... 56 segundo(s) restantes
Aguardando... 55 segundo(s) restantes
Aguardando... 54 segundo(s) restantes
Aguardando... 53 segundo(s) restantes
Aguardando... 52 segundo(s) restantes
```

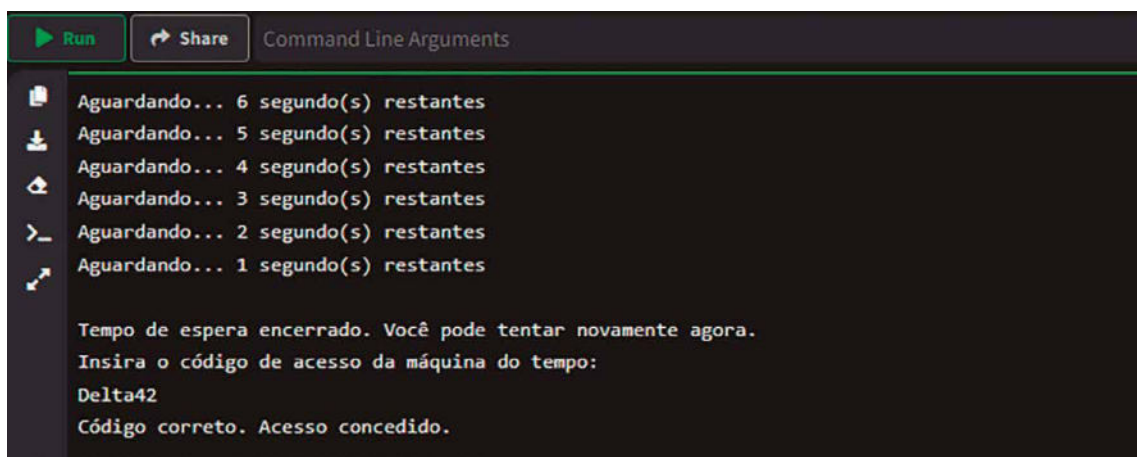
Figura 119 – Primeira tentativa de acesso frustrada à máquina do tempo



```
Run Share Command Line Arguments
Aguardando... 7 segundo(s) restantes
Aguardando... 6 segundo(s) restantes
Aguardando... 5 segundo(s) restantes
Aguardando... 4 segundo(s) restantes
Aguardando... 3 segundo(s) restantes
Aguardando... 2 segundo(s) restantes
Aguardando... 1 segundo(s) restantes

Tempo de espera encerrado. Você pode tentar novamente agora.
Insira o código de acesso da máquina do tempo:
Bazinga
Senha incorreta. Tente novamente em 2 minuto(s).
Aguardando... 120 segundo(s) restantes
Aguardando... 119 segundo(s) restantes
Aguardando... 118 segundo(s) restantes
Aguardando... 117 segundo(s) restantes
Aguardando... 116 segundo(s) restantes
```

Figura 120 – Segunda tentativa de acesso frustrada à máquina do tempo



```
Run Share Command Line Arguments
Aguardando... 6 segundo(s) restantes
Aguardando... 5 segundo(s) restantes
Aguardando... 4 segundo(s) restantes
Aguardando... 3 segundo(s) restantes
Aguardando... 2 segundo(s) restantes
Aguardando... 1 segundo(s) restantes

Tempo de espera encerrado. Você pode tentar novamente agora.
Insira o código de acesso da máquina do tempo:
Delta42
Código correto. Acesso concedido.
```

Figura 121 – Terceira tentativa de acesso à máquina do tempo, dessa vez bem-sucedida

Além da entrada pelo teclado, Python suporta outras formas de entrada de dados. Por exemplo, a leitura de arquivos permite que o programa receba informações armazenadas em documentos. Usando a função `open()`, é possível acessar arquivos de texto, ler seu conteúdo e processá-lo dentro do programa, como veremos no próximo tópico. Outra forma de entrada inclui dados provenientes de APIs ou redes, na qual bibliotecas especializadas, como `requests`, facilitam a obtenção de informações externas. Esses métodos são particularmente úteis em aplicações mais complexas, como análise de dados ou integração de sistemas.

Como aprendemos ao longo dos exercícios deste livro-texto, a função `print` em Python é uma das ferramentas mais importantes e amplamente usadas na linguagem, sendo o principal meio de exibir informações no console.

No nível básico, a função `print` recebe como argumento o que deve ser exibido no console e converte automaticamente os dados fornecidos em texto legível. Isso significa que, independentemente do tipo de dado passado para `print`, como números, strings, listas ou dicionários, a função cuidará da conversão para um formato textual adequado. Por exemplo, ao passar uma variável contendo um número inteiro, o `print` exibirá diretamente o valor numérico, sem necessidade de conversão manual. Essa funcionalidade simplifica significativamente o processo de exibição de dados em Python.

Além de exibir informações estáticas, a função `print` suporta múltiplos argumentos, separados por vírgulas, que serão exibidos em sequência no console. Por padrão, esses argumentos são separados por um espaço, mas o desenvolvedor pode modificar esse comportamento utilizando o parâmetro `sep`. Esse parâmetro é especialmente útil quando é necessário alterar o delimitador entre os itens exibidos, como substituir o espaço padrão por uma vírgula ou outro caractere. Por exemplo, exibir uma lista de números separados por vírgulas ou traços pode ser feito facilmente ajustando o valor de `sep`.

Outro recurso essencial da função `print` é o controle da terminação da linha. Por padrão, após exibir o texto, `print` adiciona uma quebra de linha (`\n`), o que move o cursor para a linha seguinte no console. Assim, cada chamada a `print` exibe informações em linhas separadas, facilitando a leitura. No entanto, esse comportamento pode ser alterado utilizando o parâmetro `end`. Com `end`, o desenvolvedor pode substituir a quebra de linha por qualquer outro caractere ou string, como um espaço ou mesmo nada. Pode ser útil em situações nas quais múltiplos `print` devem produzir uma saída contínua na mesma linha.

Ela pode ser combinada com formatações de strings, como as fornecidas por f-strings ou o método `format`, permitindo a criação de mensagens personalizadas e sofisticadas, como já fizemos diversas vezes neste livro-texto. Essa capacidade é valiosa ao exibir informações dinâmicas, como valores de variáveis ou resultados de cálculos, em mensagens claras e bem estruturadas.

6.2 Escrita de dados em arquivos

A escrita de dados em arquivos é uma funcionalidade essencial em Python, permitindo que informações sejam armazenadas de forma persistente, mesmo após o término da execução do programa. Essa capacidade é vital em uma ampla gama de aplicações, desde salvar resultados de cálculos e logs de atividades até gerar relatórios e configurar arquivos que podem ser reutilizados posteriormente.

Quando um programa escreve dados em um arquivo, ele interage diretamente com o sistema de arquivos do computador. Em Python, isso é feito por meio de operações que **abrem o arquivo** desejado, **realizam a escrita** e, por fim, **encerram a conexão** com o arquivo. Essa sequência é crucial para garantir que os dados sejam gravados corretamente e que o recurso do sistema seja liberado para outros usos. Um arquivo pode ser aberto em diferentes modos, sendo o mais comum o modo de escrita, indicado como "write". Nesse modo, caso o arquivo já exista, seu conteúdo será substituído pelos novos dados; se o arquivo não existir, ele será criado automaticamente.

A escrita em arquivos pode ser realizada de forma sequencial, na qual os dados são gravados linha por linha ou como blocos inteiros. Essa flexibilidade é útil para diferentes necessidades, como registrar

eventos contínuos em um arquivo de log ou salvar grandes quantidades de informações formatadas. Além disso, o Python oferece suporte a diferentes tipos de arquivos, incluindo arquivos de texto e binários. Arquivos de texto são usados para armazenar dados em formatos legíveis por humanos, como .txt ou .csv, enquanto arquivos binários lidam com dados estruturados ou compactados, como imagens e formatos proprietários.

Python suporta várias codificações, como UTF-8, que é amplamente utilizada para garantir compatibilidade internacional. Definir corretamente a codificação ao abrir um arquivo é essencial, especialmente ao lidar com caracteres especiais, como acentos e símbolos não ASCII.

A **escrita em arquivos** também exige cuidado com **erros e exceções** que podem ocorrer durante o processo, como permissões insuficientes, falta de espaço em disco ou falhas de hardware. Python fornece estruturas para lidar com essas situações de maneira robusta, garantindo que o programa se comporte adequadamente mesmo diante de problemas inesperados. Ao mesmo tempo, boas práticas, como o uso de blocos que gerenciam automaticamente a abertura e o fechamento de arquivos, ajudam a evitar problemas de desempenho ou inconsistências nos dados gravados.

A jornada no tempo, mesmo quando realizada sem alarde, exige um registro cuidadoso. Cada salto no continuum histórico pode revelar descobertas inestimáveis, encontros marcantes ou pequenas transformações que, no conjunto, formam o mapeamento da experiência temporal. Para não perder nada do que foi vivido, é fundamental documentar cada viagem, anotando o ano visitado e uma breve descrição do que foi encontrado, aprendido ou realizado ali. Esse registro serve como um diário pessoal do viajante do tempo e também como um repositório de dados históricos, facilitando consultas futuras e contribuindo para uma compreensão mais profunda das linhas temporais percorridas.

A seguir há um código em Python que implementa essa lógica. Ao final de cada viagem, o programa solicita ao usuário o ano que visitou e uma breve descrição do que vivenciou. Em seguida, esses dados são escritos em um arquivo de texto, garantindo a persistência das informações. Caso o arquivo não exista ainda, o Python o criará. Caso exista, novos registros serão acrescentados ao final, formando um histórico crescente de viagens.

```
# Importa a função datetime para obter a data e hora atuais do sistema.
# Embora o foco seja a viagem temporal, registrar a data atual do registro ajuda
a diferenciar quando o viajante adicionou a informação no arquivo.
from datetime import datetime

def registrar_viagem():
    # Solicita ao usuário o ano que ele visitou. Esse valor será um inteiro e
    representa o ponto no tempo ao qual o viajante se deslocou.
    # Por exemplo, se o viajante voltou para o ano 1500, inserirá esse ano para
    registrar a experiência naquele período histórico.
    ano_visitado = input("Insira o ano visitado em sua última viagem: ")

    # Solicita uma breve descrição da experiência. Aqui o usuário pode informar algo
    como "Observei uma grande feira medieval", "Conversei com um inventor do futuro"
    # ou qualquer outro detalhe que considere relevante.
    descricao = input("Descreva brevemente sua experiência nessa época: ")
```

```
# Obtém a data e hora atuais do sistema, para registrar não apenas o ano
histórico visitado, mas também em que momento, no presente, esse registro está
sendo feito.
# Isso auxilia o viajante no futuro a compreender quando adicionou cada
entrada no arquivo, correlacionando sua linha temporal pessoal com as diversas
épocas visitadas.
data_atual = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

# Abre o arquivo 'registro_viagens.txt' no modo 'a' (append). Esse modo
garante que, se o arquivo existir, o novo registro será acrescentado ao final.
# Caso o arquivo não exista, ele será criado. Ao final da operação, o 'with'
fecha o arquivo automaticamente, garantindo a integridade dos dados.
with open("registro_viagens.txt", "a", encoding='utf-8') as arquivo:
    # Escreve uma linha no arquivo contendo a data atual do registro, o ano
    visitado e a descrição.
    # O uso da formatação f-string facilita a composição da linha, tornando o
    código mais legível.
    # O caractere '\n' no final assegura que cada registro seja gravado em uma
    nova linha, mantendo o arquivo organizado.
    arquivo.write(f"{data_atual} - Ano visitado: {ano_visitado}, Descrição:
{descricao}\n")
    # Após a escrita, informa ao usuário que o registro foi realizado com
    sucesso.
    print("Registro efetuado com sucesso no arquivo 'registro_viagens.txt'.")

# Programa principal
# Neste exemplo, o código pede ao usuário se deseja fazer um registro, simula a
realização de uma viagem, e então executa a função.
# Poderíamos chamar esta função imediatamente após cada viagem efetivamente
realizada no contexto do programa maior de viagens temporais.
resposta = input("Deseja registrar uma nova viagem temporal? (s/n) ")
if resposta.lower() == 's':
    registrar_viagem()
else:
    print("Nenhum registro foi adicionado.")
```

Figura 122

Na primeira parte do código, a importação de `datetime` permite registrar o ano histórico visitado e o momento presente em que se faz o registro, trazendo contexto adicional à entrada.

A função `registrar_viagem()` é responsável por toda a lógica de coleta e armazenamento. Primeiro, obtemos o ano visitado. Aqui o programa não obriga o usuário a digitar um número estritamente, mas é possível implementar validações futuras, caso desejado, para assegurar que seja um valor coerente. Em seguida, coletamos uma descrição breve da experiência, que pode conter quaisquer detalhes, livres de qualquer formatação pré-estabelecida. A ideia é que o viajante escreva algumas palavras significativas, assim como se faz em um diário ou log de viagem.

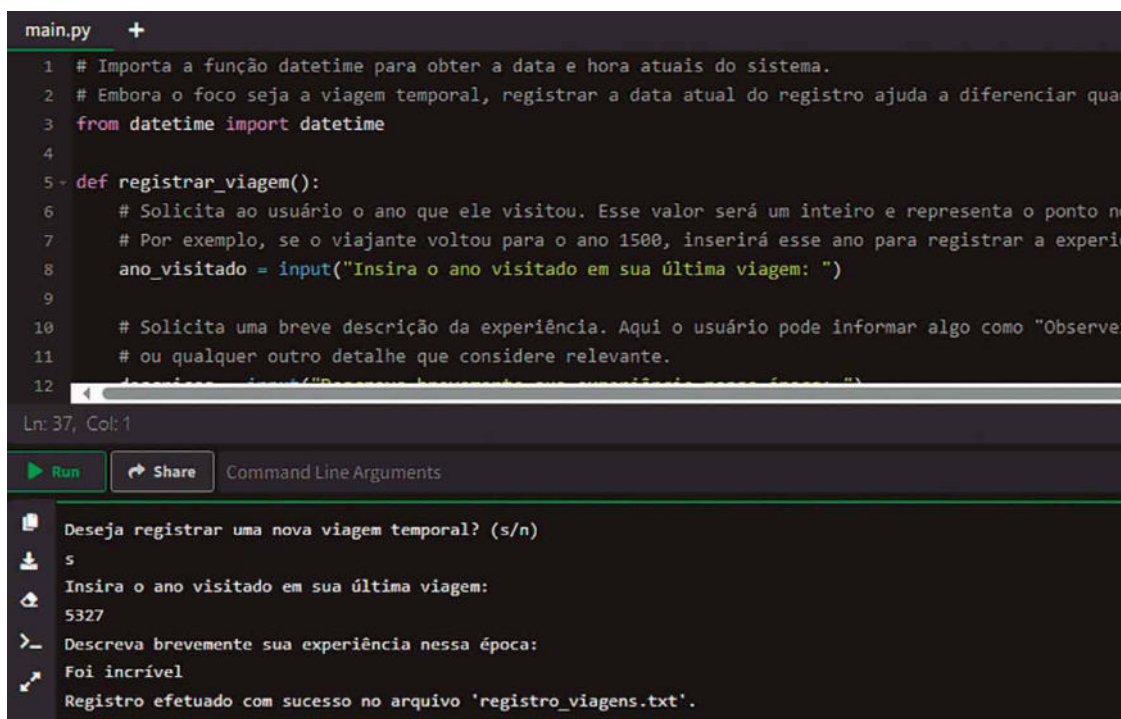
Depois, `datetime.now()` obtém a data e hora presentes, formatadas com `strftime("%Y-%m-%d %H:%M:%S")`, resultando em algo como "2030-04-15 14:30:20". Esse registro temporal contextualiza o momento em que o viajante decidiu documentar sua experiência.

O bloco `with open("registro_viagens.txt", "a", encoding='utf-8')` as `arquivo`: abre o arquivo de texto no modo `a`, garantindo que nada seja sobrescrito, apenas adicionado ao final. Caso o arquivo não exista, Python o criará automaticamente. O parâmetro `encoding='utf-8'` assegura que qualquer caractere acentuado ou símbolo especial seja corretamente armazenado, evitando problemas de codificação.

Dentro do bloco `with`, o método `arquivo.write(...)` escreve uma linha contendo a data atual, o ano visitado e a descrição, seguidos de uma quebra de linha. Ao sair do bloco `with`, o arquivo é fechado automaticamente, garantindo que o registro seja salvo adequadamente.

Após gravar a informação, o programa informa ao usuário que a operação foi um sucesso, conferindo segurança e clareza ao operador. No final, no programa principal, é solicitada uma confirmação se o usuário deseja registrar uma nova viagem. Caso responda afirmativamente, chama-se `registrar_viagem()`; caso contrário, nenhuma ação adicional é tomada. Em um cenário mais amplo, essa funcionalidade poderia ser integrada a um menu do sistema de viagens temporais, sendo chamada sempre que o viajante retornar de uma jornada no tempo, assegurando a preservação da memória histórica de suas aventuras.

Dessa forma, o código apresentado cumpre a missão de registrar dados em um arquivo, relacionando o tema de viagem no tempo a um procedimento muito prático: criar um diário histórico digital que, com o passar das viagens, se torne um verdadeiro compêndio das experiências e descobertas coletadas ao longo das eras. A figura a seguir mostra a inserção de um registro de viagem no arquivo `registro_viagens.txt`:



```
main.py +
1 # Importa a função datetime para obter a data e hora atuais do sistema.
2 # Embora o foco seja a viagem temporal, registrar a data atual do registro ajuda a diferenciar quando
3 from datetime import datetime
4
5 def registrar_viagem():
6     # Solicita ao usuário o ano que ele visitou. Esse valor será um inteiro e representa o ponto no
7     # Por exemplo, se o viajante voltou para o ano 1500, inserirá esse ano para registrar a experiência
8     ano_visitado = input("Insira o ano visitado em sua última viagem: ")
9
10    # Solicita uma breve descrição da experiência. Aqui o usuário pode informar algo como "Observei
11    # ou qualquer outro detalhe que considere relevante.
12    descricao = input("Descreva brevemente sua experiência nessa época: ")
13
14    # Escreve a data atual, o ano visitado e a descrição no arquivo de texto.
15    with open("registro_viagens.txt", "a", encoding='utf-8') as arquivo:
16        arquivo.write(f"{datetime.now().strftime('%d/%m/%Y %H:%M:%S')} | {ano_visitado} | {descricao}\n")
17
18    # Informa ao usuário que o registro foi realizado com sucesso.
19    print("Registro efetuado com sucesso no arquivo 'registro_viagens.txt'.")
20
21 # Programa principal
22 while True:
23     resposta = input("Deseja registrar uma nova viagem temporal? (s/n) ")
24     if resposta.lower() == 's':
25         registrar_viagem()
26     else:
27         break
```

Ln: 37, Col: 1

Run Share Command Line Arguments

Deseja registrar uma nova viagem temporal? (s/n)
s
Insira o ano visitado em sua última viagem:
5327
Descreva brevemente sua experiência nessa época:
Foi incrível
Registro efetuado com sucesso no arquivo 'registro_viagens.txt'.

Figura 123 – Executando o registro de viagem com escrita em arquivo

O comando `open` é uma das ferramentas mais importantes para trabalhar com arquivos, pois permite abrir arquivos existentes ou criar novos para leitura, escrita ou outras operações. Ele é o ponto de entrada principal para manipulação de arquivos na linguagem e oferece uma interface poderosa, porém simples, para interagir com o sistema de arquivos.

A função `open` aceita dois argumentos principais: o nome do arquivo e o modo de abertura. O primeiro é uma string que pode ser um caminho relativo ou absoluto. O segundo especifica o que será feito com o arquivo – por exemplo, se ele será apenas lido, escrito ou ambos. Por padrão, `open` abre arquivos no modo de leitura (`r`), e se o arquivo não existir nesse modo, ocorrerá um erro. Os modos de abertura incluem:

- `r` (leitura): abre o arquivo para leitura. Se o arquivo não existir, gera um erro.
- `w` (escrita): abre o arquivo para escrita, apagando o conteúdo existente. Se o arquivo não existir, ele será criado.
- `a` (acrescentar): abre o arquivo para adicionar conteúdo ao final. O conteúdo existente é preservado.
- `x` (criação): cria um arquivo novo, mas gera um erro se o arquivo já existir.
- `r+` (leitura e escrita): permite tanto a leitura quanto a escrita no arquivo. Ele não apaga o conteúdo existente.
- `b` (binário): adiciona suporte para trabalhar com arquivos binários, como imagens ou vídeos. Geralmente combinado com outros modos, como `rb` para leitura binária ou `wb` para escrita binária.

Um terceiro argumento opcional para `open` é o `encoding`, que é particularmente importante ao trabalhar com arquivos de texto que contenham caracteres especiais, como acentos. O padrão em Python 3 é UTF-8, mas ele pode ser alterado para atender a necessidades específicas.

O UTF-8 (abreviação de 8-bit Unicode Transformation Format) é um padrão de codificação de caracteres utilizado para representar texto em sistemas computacionais. Ele faz parte do padrão Unicode, que foi criado para unificar a representação de caracteres em diferentes idiomas e sistemas, substituindo os antigos sistemas específicos de cada língua, como ASCII ou ISO-8859.

No UTF-8, os caracteres são representados usando um ou mais bytes (8 bits por byte). Caracteres comuns em inglês, como letras e números, são codificados em um único byte, garantindo compatibilidade total com o antigo padrão ASCII. Já aqueles mais complexos, como letras com acentos, símbolos ou ideogramas chineses, usam dois, três ou mais bytes, dependendo da complexidade do caractere.



Saiba mais

Para obter mais informações sobre Python e UTF-8, você pode explorar os seguintes recursos:

A documentação oficial é a fonte mais confiável para entender como Python lida com codificações e strings. O site indicado a seguir explica detalhes sobre o uso de UTF-8, como especificar codificações ao abrir arquivos e como manipular strings Unicode:

Disponível em: <https://shre.ink/gaWI>. Acesso em: 12 dez. 2024.

O Unicode Consortium, responsável pelo padrão Unicode, tem uma ampla documentação sobre o UTF-8, explicando os princípios de design e os casos de uso. Você pode explorar mais detalhes em:

Disponível em: <https://shre.ink/gaWB>. Acesso em: 12 dez. 2024.

Fóruns como Stack Overflow frequentemente discutem problemas práticos e soluções relacionadas ao UTF-8 e ao trabalho com textos internacionais em Python:

Disponível em: <https://shre.ink/gaWd>. Acesso em: 12 dez. 2024.

Ao abrir um arquivo com `open`, ele retorna um objeto de arquivo. Esse objeto contém métodos e propriedades que permitem ler, escrever ou manipular o conteúdo do arquivo. Por exemplo, o método `read()` lê todo o conteúdo do arquivo, enquanto `write()` escreve no arquivo. Métodos como `readline()` e `readlines()` oferecem controle mais granular, permitindo ler uma linha de cada vez ou todas as linhas como uma lista.

É importante gerenciar corretamente os recursos associados ao uso de arquivos. Sempre que um arquivo é aberto, ele consome recursos do sistema, como memória, por isso é fundamental fechá-lo após o uso. Isso pode ser feito com o método `close()` do objeto de arquivo. No entanto, uma prática recomendada em Python é usar a declaração `with` junto com `open`, como fizemos no exemplo. Tal uso cria um contexto gerenciado, no qual o arquivo é **fechado automaticamente**, mesmo que ocorra um erro durante a execução.

Por exemplo, ao abrir um arquivo no modo de escrita, o comando `open("exemplo.txt", "w")` criará ou substituirá o arquivo `exemplo.txt`, retornando um objeto de arquivo que pode ser usado para escrever dados. Quando combinado com `with`, como em `with open("exemplo.txt", "w") as arquivo:`, o arquivo será automaticamente fechado ao sair do bloco `with`, garantindo a integridade dos dados e liberando os recursos adequadamente.

O comando `open` também permite manipular arquivos em diferentes diretórios, desde que o caminho seja especificado corretamente. Caminhos absolutos fornecem a localização completa no sistema de arquivos, enquanto caminhos relativos são baseados no diretório de trabalho atual do programa.

A história de uma aventura temporal não termina com o simples ato de registrar as viagens em um arquivo. Com o passar do tempo, o arquivo de registro cresce, guardando informações valiosas sobre as experiências vividas em diferentes épocas. Eventualmente, o viajante ou qualquer outro interessado pode querer consultar esses dados, revisitar as memórias descritas, estudar padrões históricos observados ou simplesmente recordar momentos marcantes. Para tal, é necessário ler o arquivo previamente criado e exibir seu conteúdo, permitindo que o viajante avalie o que já foi explorado e identifique o que ainda merece atenção.

A seguir, apresentamos um código em Python que realiza a leitura do arquivo de registro de viagens. Ele supõe que o arquivo `registro_viagens.txt` foi gerado anteriormente, como no exercício anterior, e no qual cada linha representa uma viagem, com a data do registro, o ano visitado e a descrição do ocorrido. O código verifica a existência do arquivo, tentando abri-lo. Se o arquivo não for encontrado, avisa que não há registros disponíveis. Caso exista, ele lê seu conteúdo linha a linha e exibe cada linha na tela, permitindo ao usuário navegar por sua própria história cronológica de viagens.

```
import os

def exibir_registros():
    # Antes de tentar abrir o arquivo, verifica se ele existe.
    # Isso evita erros ao tentar ler um arquivo inexistente.
    # A função os.path.exists retorna True se o caminho especificado existir,
    caso contrário False.
    if not os.path.exists("registro_viagens.txt"):
        # Caso o arquivo não exista, o programa informa que não há registros.
        print("Não há nenhum registro de viagens disponível.")
        return

    # Abre o arquivo 'registro_viagens.txt' no modo de leitura ('r').
    # Aqui, a codificação UTF-8 é indicada para garantir que caracteres
    acentuados sejam interpretados corretamente.
    with open("registro_viagens.txt", "r", encoding='utf-8') as arquivo:
        # Lê todas as linhas do arquivo. Cada linha é uma string que contém data,
        ano visitado e descrição.
        linhas = arquivo.readlines()

        # Caso o arquivo esteja vazio, significa que nenhum registro foi feito.
        Neste caso, informa-se essa situação.
        if not linhas:
            print("O arquivo de registro está vazio, nenhuma viagem foi
            registrada ainda.")
            return

        # Se o arquivo não estiver vazio, exibe cada linha para o usuário.
        # Dessa forma, é possível reler memórias, analisar padrões e refletir sobre
        cada jornada.
        print("Registros de viagens:\n")
```

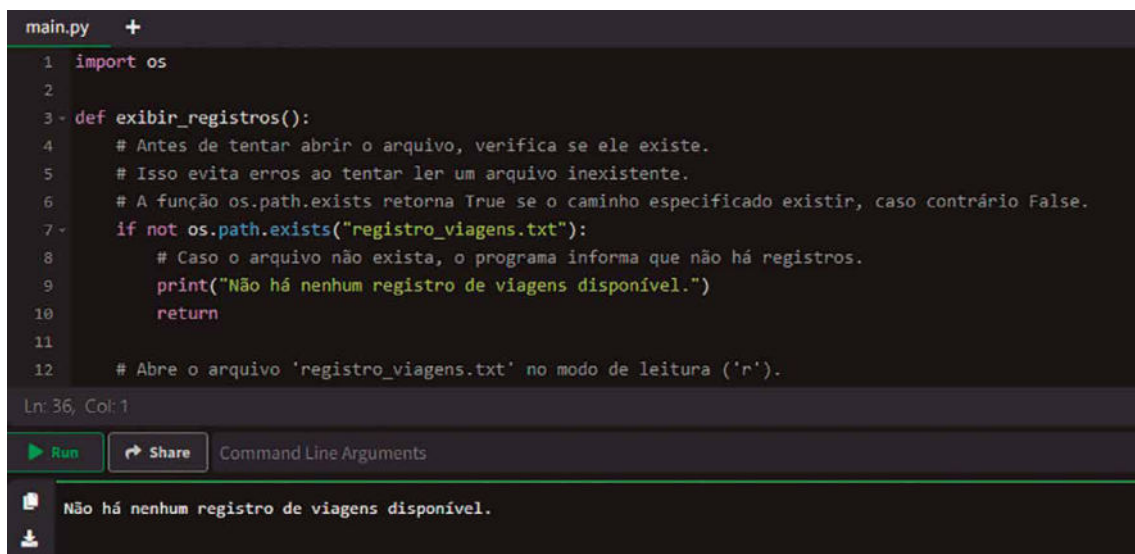


```
for linha in linhas:
    # Cada linha já contém a formatação aplicada no momento da escrita,
    logo basta imprimi-la.
    # Pode-se, se desejado, adicionar numeração ou formatação adicional
    aqui, mas não é obrigatório.
    print(linha.strip())

# Código principal:
# Neste ponto, supõe-se que o usuário ou o viajante deseje consultar os
registros.
# Ao chamar exibir_registros(), o programa tentará ler o arquivo e exibir seu
conteúdo.
# Caso não haja arquivo ou esteja vazio, mensagens informativas serão impressas.
exibir_registros()
```

Figura 124

A primeira ação é importar `os`, um módulo que fornece funções para interagir com o sistema operacional. Nesse caso, `os.path.exists` é utilizada para verificar a existência do arquivo `registro_viagens.txt`. Esse cuidado assegura que o programa não tentará ler um arquivo inexistente, o que resultaria em um erro. Em vez disso, se o arquivo não existir, imprime-se uma mensagem informando que não há registros, evidenciando ao viajante que nenhuma viagem foi gravada ou que o arquivo foi removido. A figura a seguir ilustra exatamente esse caso:



```
main.py +
1 import os
2
3 def exibir_registros():
4     # Antes de tentar abrir o arquivo, verifica se ele existe.
5     # Isso evita erros ao tentar ler um arquivo inexistente.
6     # A função os.path.exists retorna True se o caminho especificado existir, caso contrário False.
7     if not os.path.exists("registro_viagens.txt"):
8         # Caso o arquivo não exista, o programa informa que não há registros.
9         print("Não há nenhum registro de viagens disponível.")
10        return
11
12    # Abre o arquivo 'registro_viagens.txt' no modo de leitura ('r').

Ln: 36, Col: 1
Run Share Command Line Arguments
Não há nenhum registro de viagens disponível.
```

Figura 125 – O registro de viagens não está disponível no caminho indicado

Em seguida, abre-se o arquivo no modo de leitura (`r`) e com codificação `utf-8`. O `with` garante que o arquivo seja fechado automaticamente ao final do bloco, evitando vazamentos de recursos. Em `linhas = arquivo.readlines()`, todas as linhas do arquivo são carregadas em memória, formando uma lista de strings. Cada linha representa um registro de uma viagem, conforme o padrão estabelecido anteriormente, contendo data do registro, ano visitado e descrição.

Se a lista `linhas` estiver vazia, significa que o arquivo existe, mas não possui conteúdo útil. Nesse caso, o programa informa que não há viagens registradas. Isso pode ocorrer, por exemplo, se o arquivo foi criado mas ainda nenhum registro foi escrito nele.

Caso haja linhas no arquivo, o programa entra em um loop `for linha in linhas:`, imprimindo cada registro. O uso de `linha.strip()` remove eventuais espaços em branco e quebras de linha extras, garantindo uma saída mais limpa. Ao imprimir cada linha, o usuário pode reler seus registros, estudar suas rotas temporais anteriores e refletir sobre as descobertas feitas. Se desejar, pode-se incluir formatação adicional, como numerar os registros ou imprimir separadores entre eles.

A máquina do tempo, sendo um dispositivo complexo, não depende apenas de um simples ano de destino para funcionar. Ela tem um conjunto de configurações, parâmetros e preferências que podem ser definidos pelo operador para determinar a forma como os saltos temporais serão realizados. É possível que o viajante queira ajustar a quantidade de energia inicial, a sensibilidade do controle de destino, o modo de viagem (gradual ou instantâneo) ou mesmo habilitar ou desabilitar recursos de segurança. Todas essas configurações formam o estado interno da máquina, um conjunto de dados que, se perdido, obrigaria o operador a redefinir tudo manualmente a cada início do sistema. Uma solução elegante é manter um arquivo de backup das configurações, permitindo salvá-las em um momento e restaurá-las posteriormente, garantindo a continuidade e a coerência da experiência de viagem no tempo.

A seguir, há um código Python que implementa esse conceito. Ele salva as configurações da máquina do tempo em um arquivo de texto usando o formato JSON, uma forma padronizada de representar dados. Depois, é possível carregar essas configurações novamente. Esse procedimento assegura que as preferências do operador não sejam perdidas entre execuções do programa ou eventuais falhas.

```
import json
import os

def salvar_configuracoes(configuracoes, nome_arquivo="configuracoes.json"):
    # Esta função recebe um dicionário 'configuracoes' contendo as chaves e
    # valores das preferências do operador.
    # Por exemplo, {'modo_de_viagem': 'gradual', 'energia_inicial': 5000,
    # 'sensibilidade_controle': 0.8}.
    # O objetivo é gravar esses dados em um arquivo JSON, permitindo
    # restaurá-los futuramente.

    # Abre (ou cria, caso não exista) o arquivo especificado no modo escrita
    # ('w').
    # O parâmetro encoding='utf-8' garante que caracteres especiais sejam
    # armazenados corretamente.
    with open(nome_arquivo, "w", encoding='utf-8') as arquivo:
        # Usa a função json.dump para converter o dicionário em uma string JSON e
        # salvá-lo no arquivo.
        # O parâmetro indent=4 torna o JSON mais legível, alinhando as chaves e
        # valores em múltiplas linhas.
        json.dump(configuracoes, arquivo, indent=4)

    # Após a gravação, informa que o backup foi concluído, ajudando o usuário a
    # perceber que as configurações estão seguras.
```

```
print(f"Configurações salvas com sucesso em {nome_arquivo}.")

def carregar_configuracoes(nome_arquivo="configuracoes.json"):
    # Esta função tenta carregar as configurações a partir de um arquivo JSON.
    # Caso o arquivo não exista ou esteja vazio, o programa pode criar um
    # dicionário padrão.

    # Verifica se o arquivo existe. Caso não exista, retorna um dicionário vazio
    # ou com valores padrão.
    if not os.path.exists(nome_arquivo):
        # Não há backup prévio, retorna configurações padrão.
        print("Nenhum arquivo de configuração encontrado. Usando valores padrão.")
        return {}

    # Se o arquivo existir, abre-o no modo leitura ('r') e carrega as
    # configurações.
    with open(nome_arquivo, "r", encoding='utf-8') as arquivo:
        # Usa a função json.load para ler o conteúdo do arquivo e convertê-lo de
        # volta em um dicionário.
        configuracoes = json.load(arquivo)

    # Informa o usuário que as configurações foram carregadas, para que saiba que
    # os valores retornados refletem um estado anterior.
    print(f"Configurações carregadas com sucesso de {nome_arquivo}.")
    return configuracoes

# Código principal de exemplo:
# Suponhamos que o operador da máquina do tempo queira ajustar algumas
# preferências antes de iniciar a viagem.
# Por exemplo, definir o modo de viagem como 'gradual', a energia inicial da
# máquina, a sensibilidade do controle,
# e ainda marcar se recursos de segurança avançados devem ser habilitados.
config_iniciais = {
    "modo_de_viagem": "gradual",
    "energia_inicial": 5000,
    "sensibilidade_controle": 0.8,
    "recursos_seguranca": True
}

# Salva as configurações iniciais no arquivo.
salvar_configuracoes(config_iniciais)

# Mais tarde, quando o programa é executado novamente ou após um reinício da
# máquina, o operador deseja recuperar as configurações.
# Isso assegura que não será necessário redefinir tudo manualmente.
config_restauradas = carregar_configuracoes()

# Exibe as configurações restauradas para confirmar que foram lidas corretamente
# do arquivo.
print("Configurações atuais da máquina do tempo:", config_restauradas)
```

Figura 126

O código apresentado implementa um sistema simples e eficiente para salvar e carregar configurações utilizando arquivos no formato JSON, um dos mais populares para o armazenamento e intercâmbio

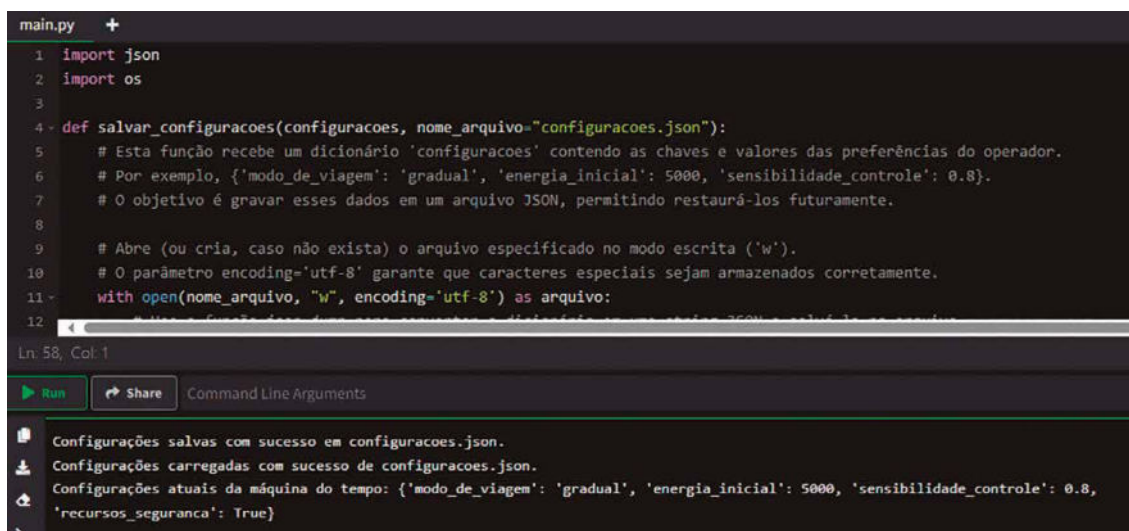
de dados estruturados. Ele se divide em duas funções principais: `salvar_configuracoes` e `carregar_configuracoes`, ambas operando sobre arquivos JSON para garantir que as configurações sejam persistentes entre execuções do programa.

A função `salvar_configuracoes` é responsável por armazenar as configurações fornecidas em um arquivo JSON. Para isso, ela recebe um dicionário Python contendo as configurações a serem salvas e um nome de arquivo, que, por padrão, é `configuracoes.json`. Ao abrir o arquivo no modo de escrita (`w`), ela utiliza a biblioteca `json` para converter o dicionário em um formato JSON, que é compatível com sistemas externos e humano-legível quando formatado corretamente. O parâmetro `indent=4` na função `json.dump` adiciona espaços de recuo para tornar o arquivo mais fácil de interpretar, alinhando as chaves e os valores. Após a gravação, uma mensagem é exibida para informar que as configurações foram salvas com sucesso, dando ao usuário a confiança de que as preferências estão seguras.

Por outro lado, a função `carregar_configuracoes` é projetada para recuperar essas configurações do arquivo JSON previamente salvo. Ela começa verificando se o arquivo especificado existe no sistema utilizando a função `os.path.exists`. Caso o arquivo não esteja presente, a função retorna um dicionário vazio ou um conjunto de valores padrão, indicando ao usuário que nenhuma configuração anterior foi encontrada. No entanto, se o arquivo existir, ele é aberto no modo de leitura (`r`), e seu conteúdo é carregado de volta para um dicionário Python usando a função `json.load`. Desse modo, as configurações salvas são restauradas de forma simples e direta, garantindo que o estado do programa seja preservado. Uma mensagem é exibida após o carregamento para informar que as configurações foram recuperadas com sucesso.

No exemplo principal do código, o sistema é utilizado para gerenciar as preferências de uma máquina do tempo fictícia. Inicialmente, as configurações, como `modo_de_viagem`, `energia_inicial` e outras opções, são definidas em um dicionário chamado `config_iniciais`. Esse dicionário é passado para a função `salvar_configuracoes`, que grava suas informações em um arquivo JSON. Posteriormente, ao simular o reinício do programa, as configurações são carregadas de volta utilizando a função `carregar_configuracoes`, garantindo que o estado anterior seja recuperado sem a necessidade de redefinições manuais. Por fim, o programa exibe as configurações restauradas para confirmar que o processo foi concluído corretamente.

Esse código demonstra várias boas práticas no gerenciamento de dados persistentes. Ele utiliza o formato JSON, que é amplamente suportado e legível tanto por humanos quanto por máquinas. Além disso, o uso da declaração `with` ao abrir os arquivos garante que eles sejam fechados automaticamente após o uso, evitando problemas de recursos do sistema. A separação em funções modulares também melhora a legibilidade e reutilização do código, facilitando sua expansão para incluir outras configurações ou funcionalidades no futuro. No geral, esse sistema de salvar e carregar configurações é útil em muitos contextos, como aplicativos, sistemas de controle e ferramentas personalizadas que precisam manter estados entre execuções. A figura a seguir apresenta a execução bem-sucedida do programa:



```
main.py +
1 import json
2 import os
3
4 def salvar_configuracoes(configuracoes, nome_arquivo="configuracoes.json"):
5     # Esta função recebe um dicionário 'configuracoes' contendo as chaves e valores das preferências do operador.
6     # Por exemplo, {'modo_de_viagem': 'gradual', 'energia_inicial': 5000, 'sensibilidade_controle': 0.8}.
7     # O objetivo é gravar esses dados em um arquivo JSON, permitindo restaurá-los futuramente.
8
9     # Abre (ou cria, caso não exista) o arquivo especificado no modo escrita ('w').
10    # O parâmetro encoding='utf-8' garante que caracteres especiais sejam armazenados corretamente.
11    with open(nome_arquivo, "w", encoding='utf-8') as arquivo:
12        # Grava o dicionário em formato JSON no arquivo especificado.
13        json.dump(configuracoes, arquivo)
14
15 # Exemplo de uso:
16 configuracoes = {'modo_de_viagem': 'gradual', 'energia_inicial': 5000, 'sensibilidade_controle': 0.8, 'recursos_seguranca': True}
17 salvar_configuracoes(configuracoes)
```

Ln: 58, Col: 1

Run Share Command Line Arguments

Configurações salvas com sucesso em configuracoes.json.
Configurações carregadas com sucesso de configuracoes.json.
Configurações atuais da máquina do tempo: {'modo_de_viagem': 'gradual', 'energia_inicial': 5000, 'sensibilidade_controle': 0.8, 'recursos_seguranca': True}

Figura 127 – Salvando um backup das configurações da máquina do tempo em arquivo JSON.
O programa também apresenta uma restauração do backup na sequência

Em resumo, a escrita de dados em arquivos é um aspecto essencial da programação em Python, permitindo a persistência de informações e a interação do programa com o sistema de arquivos. Essa funcionalidade é implementada de maneira simples e eficaz, tornando-a acessível tanto para iniciantes quanto para programadores experientes. A capacidade de armazenar dados de forma persistente amplia as possibilidades de aplicações Python, tornando essa linguagem ideal para tarefas que envolvem manipulação de dados em arquivos.



Resumo

Esta unidade explorou em profundidade o uso e a manipulação de listas e dicionários em Python, apresentando-os como estruturas essenciais para o gerenciamento e organização de dados em programação. As listas são descritas como sequências ordenadas e mutáveis, permitindo armazenar e manipular coleções de elementos de forma flexível. Os dicionários, por sua vez, são acentuados como estruturas baseadas em pares chave-valor, ideais para acessar e organizar informações de forma eficiente.

No que diz respeito às listas, detalhamos operações fundamentais como adição, remoção, ordenação e fatiamento de elementos. A flexibilidade dessas operações foi ilustrada em cenários práticos, como a criação de listas de destinos temporais para viagens no tempo, nas quais é possível organizar, modificar e consultar os dados de forma dinâmica.

Os dicionários foram destacados como ferramentas eficientes para representar relações entre dados, permitindo acesso direto a valores por meio de chaves. Ilustramos aplicações práticas, como o armazenamento de perfis de viajantes do tempo e a criação de agendas temporais que associam anos a compromissos específicos. Além disso, abordamos o uso de dicionários aninhados para registrar eventos históricos com detalhes adicionais, demonstrando como essa estrutura pode ser utilizada para organizar informações hierarquicamente.

Abordamos detalhadamente conceitos vitais e avançados relacionados às operações de entrada e saída de dados em Python, estudando sua aplicação em diferentes contextos de programação. A interação entre o programa e o ambiente externo, seja com o usuário, outros sistemas ou dispositivos, é apresentada como um componente essencial para a funcionalidade de softwares.

A entrada de dados é discutida principalmente por meio da função `input()`, que permite capturar informações inseridas pelo usuário. Exploramos a natureza dessa entrada como strings e a necessidade de conversão para tipos apropriados, como inteiros ou floats, dependendo do uso. O uso de validações para evitar entradas inválidas foi enfatizado, garantindo robustez nos programas.

Além das operações básicas de entrada e saída, ressaltamos a manipulação de arquivos como um método para persistir dados além da execução do programa. Foram discutidos modos de abertura de arquivos,

como leitura, escrita e adição de conteúdo, além de cuidados necessários para evitar erros, como a manipulação inadequada de codificações de caracteres. A persistência de dados foi exemplificada com o registro de viagens temporais em arquivos de texto em formato JSON, permitindo ao usuário manter um histórico de suas interações.



Exercícios

Questão 1. Considere o código Python a seguir, que faz parte de um aplicativo para análise de atividades de usuários. O código filtra usuários com base nas suas atividades e, em seguida, manipula essas atividades de modo a reorganizá-las e extrair uma lista única de ações.

```
app_data = {
    "usuarios": {
        "joao": ["login", "checkout", "logout"],
        "maria": ["login", "search", "add_to_cart", "logout"],
        "pedro": ["login", "search", "search", "logout"]
    },
    "status": "active"
}

# Filtrar usuários cujas listas de atividades contenham "search" mais de uma vez
usuarios_filtrados = {}
for nome, atividades in app_data["usuarios"].items():
    if atividades.count("search") > 1:
        usuarios_filtrados[nome] = atividades

# Remover "logout" final (se existir) e adicionar o nome do usuário no início da lista
for nome, atividades in usuarios_filtrados.items():
    if atividades[-1] == "logout":
        atividades.pop()
    atividades.insert(0, nome)

# Criar uma lista única de atividades (exceto o nome do usuário), mantendo a ordem de primeira aparição
atividades_unicas = []
for atividades in usuarios_filtrados.values():
    for item in atividades[1:]:
        if item not in atividades_unicas:
            atividades_unicas.append(item)

print(usuarios_filtrados)
print(atividades_unicas)
```

Figura 128

Analise o código e assinale a alternativa que mostra corretamente a saída final impressa no console ao término da execução.

- A) {'pedro': ['pedro', 'login', 'search', 'search']} impresso para usuarios_filtrados e ['login', 'search'] impresso para atividades_unicas.
- B) {'maria': ['maria', 'login', 'search', 'add_to_cart']} impresso para usuarios_filtrados e ['login', 'search', 'add_to_cart'] impresso para atividades_unicas.

C) {'pedro': ['pedro', 'search', 'search']} impresso para usuarios_filtrados e ['search'] impresso para atividades_unicas.

D) {'joao': ['joao', 'login', 'checkout']} e {'pedro': ['pedro', 'login', 'search', 'search']} para usuarios_filtrados, e ['login', 'checkout', 'search'] para atividades_unicas.

E) {} para usuarios_filtrados e [] para atividades_unicas.

Resposta correta: alternativa A.

Análise da questão

Ao analisar o código, é preciso compreender passo a passo o que está acontecendo com o dicionário `app_data`. Inicialmente, há um conjunto de usuários, cada um com uma lista de atividades. O primeiro loop percorre `app_data["usuarios"]`, verificando quantas vezes `search` aparece na lista de cada usuário. O critério de filtragem é contar quantas vezes `search` ocorre; se for mais de uma vez, o usuário e sua lista são copiados para `usuarios_filtrados`. No caso apresentado, `joao` tem a lista `["login", "checkout", "logout"]` sem `search`; `maria` tem apenas um `Search` e, portanto, não atende ao critério. Já `pedro` apresenta `["login", "search", "search", "logout"]`, contendo `search` duas vezes e cumprindo, assim, o requisito. Dessa forma, após o primeiro loop, teremos `usuarios_filtrados = {"pedro": ["login", "search", "search", "logout"]}`.

Em seguida, o segundo loop percorre `usuarios_filtrados` e manipula as listas. Ele verifica se o último elemento é `logout` e, se for, remove-o. Para `pedro`, a lista se torna `["login", "search", "search"]`. Depois, há a inserção do nome do usuário no índice zero, resultando em `["pedro", "login", "search", "search"]`. Agora, temos `usuarios_filtrados = {"pedro": ["pedro", "login", "search", "search"]}`.

Por fim, o último trecho do código cria a lista `atividades_unicas`. Ele percorre as listas dos usuários filtrados, ignorando o primeiro elemento (que agora é o nome do usuário) e coletando apenas as atividades subsequentes, inserindo cada uma na nova lista se ainda não estiver presente, mantendo assim a ordem de primeira aparição.

A única lista após o filtro é a de `pedro`, que fornece as atividades `login`, `search` e `search`. O primeiro `login` entra em `atividades_unicas`, resultando em `["login"]`, depois `search` entra, resultando em `["login", "search"]`. A terceira ocorrência de `search` não adiciona nada, pois já existe. Dessa forma, obtemos `atividades_unicas = ["login", "search"]`.

Portanto, a saída impressa é: `usuarios_filtrados = {'pedro': ['pedro', 'login', 'search', 'search']}` e `atividades_unicas = ['login', 'search']`. Essa saída corresponde exatamente à alternativa A.

Questão 2. Considere um pequeno aplicativo de uma farmácia online no qual o farmacêutico registra pedidos de clientes para posterior processamento. O código a seguir, em Python, lê o nome de cada medicamento e as instruções de uso (como dosagem, posologia etc.) digitados pelo usuário no terminal. Caso o mesmo medicamento seja informado mais de uma vez, o código agrupa as diferentes instruções em uma lista associada àquela chave. Ao término da entrada dos dados, o programa grava todas as informações em um arquivo JSON, representando o registro do pedido.

```
import json

pedido = {}
while True:
    medicamento = input("Nome do medicamento (Enter para finalizar): ")
    if medicamento == "":
        break
    instrucao = input("Instruções para '{}': ".format(medicamento))

    if medicamento in pedido:
        if isinstance(pedido[medicamento], list):
            pedido[medicamento].append(instrucao)
        else:
            pedido[medicamento] = [pedido[medicamento], instrucao]
    else:
        pedido[medicamento] = instrucao

with open("pedido.json", "w", encoding="utf-8") as f:
    json.dump(pedido, f, ensure_ascii=False, indent=2)
```

Figura 129

Considere também o diagrama de sequência apresentado a seguir, que ilustra a interação passo a passo entre o usuário e o sistema.

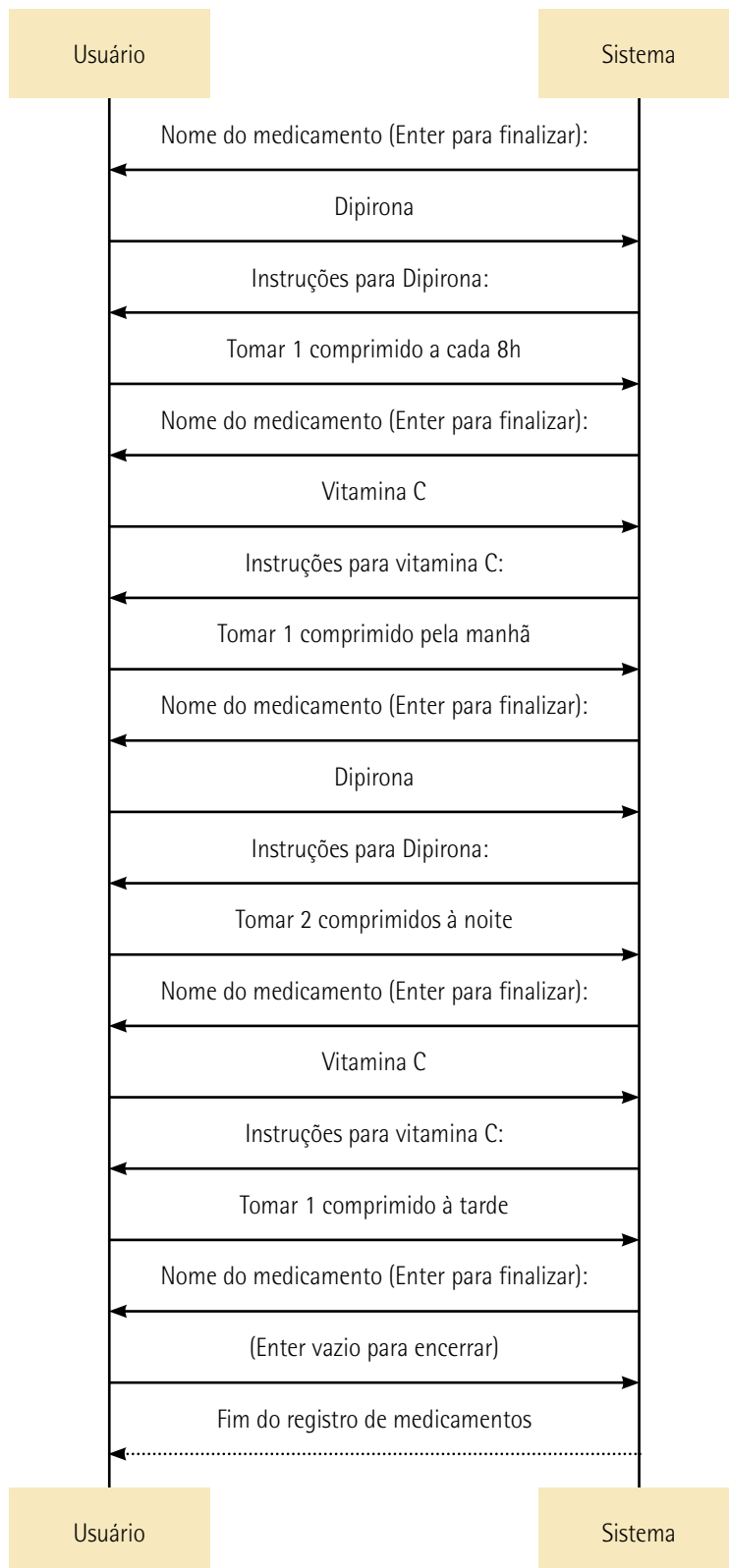


Figura 130

Analise o código, considere as entradas fornecidas e assinale a alternativa que descreve corretamente o conteúdo final do arquivo `pedido.json`:

A) O arquivo `pedido.json` conterá:

```
{  
  "Dipirona": "Tomar 2 comprimidos à noite",  
  "Vitamina C": "Tomar 1 comprimido à tarde"  
}
```

B) O arquivo `pedido.json` conterá:

```
{  
  "Dipirona": ["Tomar 1 comprimido a cada 8h", "Tomar 2 comprimidos à noite"],  
  "Vitamina C": "Tomar 1 comprimido pela manhã"  
}
```

C) O arquivo `pedido.json` conterá:

```
{  
  "Dipirona": ["Tomar 1 comprimido a cada 8h", "Tomar 2 comprimidos à noite"],  
  "Vitamina C": ["Tomar 1 comprimido pela manhã", "Tomar 1 comprimido à tarde"]  
}
```

D) O arquivo `pedido.json` conterá:

```
{  
  "Dipirona": ["Tomar 2 comprimidos à noite"],  
  "Vitamina C": ["Tomar 1 comprimido à tarde"]  
}
```

E) O arquivo `pedido.json` conterá:

```
{  
}
```

Resposta correta: alternativa C.

Análise da questão

O código apresentado realiza leituras sucessivas do nome de um medicamento e das suas respectivas instruções de uso. Caso o usuário pressione Enter sem digitar o nome de um medicamento, o loop é encerrado e o dicionário `pedido` é gravado em `pedido.json`. É fundamental analisarmos o momento em que cada medicamento é inserido no dicionário e como o código lida com a repetição.

Como vemos no diagrama de sequência, assim que o sistema inicia a solicitação de digitação do nome do primeiro medicamento, o usuário digita Dipirona e, depois que o sistema solicita as instruções para a Dipirona, o usuário digita Tomar 1 comprimido a cada 8h. Como Dipirona não existia no dicionário pedido, é criada a entrada {"Dipirona": "Tomar 1 comprimido a cada 8h"}. Em seguida, após nova solicitação de digitação de nome de medicamento feita pelo sistema, o usuário insere Vitamina C com Tomar 1 comprimido pela manhã. Agora, o dicionário contém {"Dipirona": "Tomar 1 comprimido a cada 8h", "Vitamina C": "Tomar 1 comprimido pela manhã"}.

Quando Dipirona é digitada novamente, agora com o valor Tomar 2 comprimidos à noite, o código detecta que Dipirona já existe e não é uma lista. Então, converte o valor da chave para uma lista, preservando o valor anterior. Dessa forma, Dipirona passa a ser ["Tomar 1 comprimido a cada 8h", "Tomar 2 comprimidos à noite"]. O dicionário pedido agora é {"Dipirona": ["Tomar 1 comprimido a cada 8h", "Tomar 2 comprimidos à noite"], "Vitamina C": "Tomar 1 comprimido pela manhã"}.

Após isso, insere-se Vitamina C novamente, dessa vez com Tomar 1 comprimido à tarde. Como Vitamina C já existe e não é uma lista, o código converte o valor em lista, preservando o primeiro valor e acrescentando o segundo. Agora, Vitamina C passa a ser ["Tomar 1 comprimido pela manhã", "Tomar 1 comprimido à tarde"]. Assim, o dicionário final antes da gravação fica {"Dipirona": ["Tomar 1 comprimido a cada 8h", "Tomar 2 comprimidos à noite"], "Vitamina C": ["Tomar 1 comprimido pela manhã", "Tomar 1 comprimido à tarde"]}

A figura a seguir resume a evolução do dicionário pedido após cada interação:

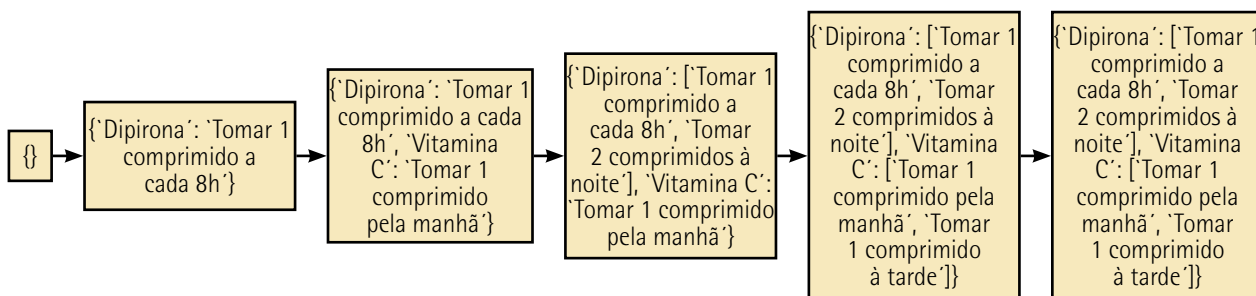


Figura 131

Quando o usuário pressiona Enter sem digitar um novo medicamento, o loop se encerra e o dicionário é escrito em formato JSON exatamente nessa forma, o que corresponde integralmente à alternativa C.